

Improving the Operating Stability of Magnetically Joinable Continuum Robots

by

Adrian Shkumbov

Supervisor: Eric Diller

April 7th, 2026

B.A.Sc. Thesis



Division of Engineering Science
UNIVERSITY OF TORONTO

Abstract

Continuum robots (CRs) offer distinct advantages in navigating confined spaces due to their physical compliance, providing potential for applications in minimally invasive surgery. However, this same characteristic inherently limits their ability to execute force application tasks. To address this limitation, magnetic joining through embedded passive permanent magnets presents an attractive method for modulating stiffness, allowing individual CRs to join and brace against one another without relying on bulky or complicated mechanisms. However, the unjoining process of such CRs through naive actuation introduces unstable "snapping behavior," which causes the robots to overshoot and violate procedural safety margins. This work investigates the effectiveness of a novel control scheme designed to minimize snapping behavior in a tendon driven magnetic CR. The proposed approach utilizes active tendon compensation driven by high-speed inertial feedback. To achieve this, a custom 10 mm diameter tip-mounted inertial measurement unit (IMU) sensor board was developed, providing rapid acceleration-based feedback to detect, trigger, and inform responses to snapping events in a compact form factor. Experimental testing was conducted on a magnetic CR restricted to planar motion unjoining from a fixed set of magnets. It is shown that through the execution of predetermined tendon compensation profiles, the system successfully counteracts elastic build-up, reducing snapping overshoot to 3 mm from a 25 mm baseline response. These results were replicated with IMU-triggered compensation, wherein the unjoining procedure is interrupted when localized acceleration thresholds are exceeded. The work concludes with the evaluation of a feedback controller formulated for the planar test setup.

Acknowledgements

First, I would like to thank my supervisor, Professor Eric Diller. I'm grateful for his guidance, support, and the freedom he gave me with regards to this project's direction. I would also like to thank Professor Jessica Burgner-Kahrs for her input and kind words of encouragement, as well as the members of the Continuum Robotics Laboratory, who provided valuable feedback in the early stages of this work. A special thanks goes to my mentor Ella Walsh, who has been my biggest supporter in all this. Her constant aid and thoughtful feedback throughout this project have been crucial. With regards to PCB development, I'd like to thank Stephen Yang for checking over my designs. Finally, I'd like to thank Serena Beddoe for helping me assemble the parts of this work's test setup difficult for my "large and oafish" hands. Whatever success I've achieved here is underpinned by all this support that I've received, and for that I'm truly grateful.

Contents

Abstract	i
Acknowledgements	ii
1 Introduction	1
1.1 Motivation	1
1.2 Approach & Research Objectives	2
2 Background	3
2.1 Tendon Driven Continuum Robots	3
2.1.1 Stiffening Approaches	5
2.1.2 Constant Curvature and Piecewise Constant Curvature Modeling . .	6
2.1.3 Sensing	8
2.2 Vibrations in TDCRs & Similar Cable Driven Systems	9
2.3 IMUs in Vibration Suppression Systems	10
3 Methods	12
3.1 Problem Simplification	12
3.2 IMU Sensor Board Development	13
3.2.1 Robot Vibration Frequencies	13
3.2.2 IMU Selection	16
3.2.3 PCB Design	18
3.3 Final Experimental Setup & IMU Integration	19
4 Testing & Results	21
4.1 Baseline Snapping Response	21
4.2 Open-loop Tendon Compensation	22
4.2.1 Uniform Tensioning in Response to Snapping	22
4.2.2 Execution of Pre-programmed Compensation Profiles	23
4.3 Incorporation of IMU Feedback	25
4.3.1 IMU Triggered Compensation Profiles	25
4.3.2 Feedback Control with IMU Data	26
5 Conclusion	30
References	32
A IMU Sensor Board Design Information	35
A.1 Wiring Diagram	35
A.2 PCB Layout	36
A.3 Sensor Polling Firmware	37

B	Test Setup Hardware and Software	39
B.1	Hardware Utilized	39
B.2	Computer Software	39

List of Figures

1	Diagram of a Typical TDCR [11]	4
2	Diagram Depicting CC Bending [14]	6
3	Examples of CDPRs [21, 20]	10
4	Diagram of Simplified Snap Testing Configuration	13
5	Planar Test Setup	14
6	Example Frame of Tracking Footage (Top) & Tip Position Estimate (Bottom)	14
7	Fitted Curve Against Free Vibration Data	15
8	Software Rendering of Sensor Board Design	18
9	Manufactured Sensor Board	19
10	Experimental Setup Component Connections	20
11	Final Experimental Setup	20
12	Baseline Snapping Response of Experimental Setup	21
13	Captured IMU Data from 2 Consecutive Baseline Snaps	22
14	Effects of Robot Tensioning During Snapping	23
15	Robot Unjoining with Tailored Compensation Profile	24
16	Effects of Robot Tensioning During Snapping	25
17	Robot Unjoining with IMU Triggered Compensation Profile	26
18	Relative Sensitivities of Segment Curvatures on Tip Position $\frac{\partial x}{\partial \kappa_1} : \frac{\partial x}{\partial \kappa_2}$	28
19	Sample Outputs of Feedback Controller Tests	29
20	PCB Wiring Diagram	35
21	PCB Layout	36

List of Tables

1	Results of Curve Fit	16
2	LSM6DSV16X IMU Specifications [24] vs. Application Requirements	17
3	Experimental Setup Hardware Components	39

1 Introduction

1.1 Motivation

Continuum robots (CRs) are slender compliant mechanisms capable of navigating through narrow tortuous passages and operating in confined spaces. These characteristics give rise to their numerous applications in which traditional rigid-jointed robots would struggle to operate. A notable example of this and the context of this work is in robotic surgery, where CRs have been proposed/developed to perform minimally invasive procedures in various hard-to-reach places within the body [1]. However, this compliance of CRs which serves as advantage in navigation and manipulation, quickly becomes a disadvantage in force application tasks. Due to their long compliant nature, CRs often suffer unwanted deflections/bending in response to tasks such as driving needles or cutting tissue, diminishing their effectiveness. This has led to many efforts to increase the stiffness of CRs when required while also preserving their manipulability during navigation [2]. One up-and-coming method of interest is to join two CRs, increasing stiffness through bracing [3]. With this approach comes the added possibility of multiple tool handling and other collaborative functions for complex tasks, which are highly desirable in medical applications [4].

Small embedded permanent magnets are a prime candidate to facilitate CR joining due to their safety, relative ease of actuation, and ease of miniaturization. They are commonly used as an actuation strategy in similar medical devices such as steerable catheters for like reasons [5]. However, rather than using embedded magnets and external magnetic fields for direct actuation as in the steerable catheter, a research goal is to use them as a secondary actuation mode to allow for robot joining through magnetic attraction [3]. This would remove the need for dedicated joining mechanisms which add complexity and are difficult to integrate within the tight size constraints of surgical devices.

Unfortunately, manipulating magnetic continuum robots alongside each other poses significant challenges involving unwanted magnetic interactions. Most notably, detaching a magnetic join through naive actuation causes abrupt and erratic motions/vibrations, which will be referred to as **snaps/snapping** here on in [3]. In order for magnetically joinable CRs to be viable, this snapping behavior must be minimized.

1.2 Approach & Research Objectives

This thesis aims to investigate snapping behavior in magnetic CRs, with the goal to improve their operating stability so that they can be effectively used for joining without erratic behavior. In particular, a magnetic continuum robot design and accompanying control scheme is sought which minimizes snapping behavior for effective operation. To investigate this, this work will focus on the development and testing of a magnetic CR setup constituting a simplified version of the full snapping problem, whereby a magnetic CR restricted to planar motion undergoes joining/unjoining with a fixed set of magnets. This is designed to allow for the analysis and development of fundamental mitigation methods for a basic case of snapping, which may be furthered in future work. A full description of this experimental construction can be found in section 3.1.

The use of active tendon compensation with inertial feedback via a tip-mounted inertial measurement unit (IMU) is the chosen approach to mitigate snapping. This seeks to build upon previous work in tendon driven continuum robot (TDCR) control and CR sensing, as well as in vibration suppression of similar cable-driven systems. The design philosophy behind this approach is a snapping suppression system which minimally changes the design of a magnetic TDCR, keeping size down through using it's means of actuation as the source of snapping compensation with feedback from a compact low-cost sensor. Section 2 details how existing literature supports this proposed technique, and its advantages.

In accordance with the proposed approach, the core research objectives of this work are as follows:

- (A) Determine the feasibility of tendon compensation as a control strategy for damping snapping behavior.
- (B) Explore the use of inertial feedback for fast and accurate detections of snapping.
- (C) Integrate active tendon compensation with IMU feedback and evaluate the effectiveness of the integrated system.

Tests will comprise evaluations of different control strategies in the context of specific unjoining scenarios and target actions. The key metrics to be used in this work are as follows.

- (I) Snapping-caused overshoot (mm).
- (II) Achievable IMU data rate (Hz) and consistency in snapping signal detection.

(III) Achievable control loop rate (Hz) and response settling time (ms)

While the maximum allowable involuntary displacement/overshoot as a result of snapping in a surgical scenario is difficult to pinpoint and would depend on the specific instance of use, typical safety margins for minimally invasive procedures are on the order of a few (3-5) millimeters [6, 7]. Consequently, the "safe limit" that will be targeted for metric I is a snapping overshoot of less than 5 mm.

In terms of metric II, the literature indicates that for vibration suppression applications, data rates an order of magnitude or greater above the highest vibration mode considered produce strong results [8, 9]. The latter half of this metric refers to the ability of the sensor to capture a distinct signal indicative of snapping free of corrupting noise. These are the main guiding principles for this work's sensor board development and integration, and are further discussed in section 3.2. In a similar vein, metric III also aims for a similar frequency advantage compared to the observed snapping frequency, with the ultimate action rate built on top of incoming sensor measurements being the decider of effective compensation [9]. In addition, it is desirable for the utilized control method to settle the robot's response in a timely manner to continue regular operation, with less than one period of the robot's free vibration being targeted for this metric.

2 Background

In this section, core information on tendon driven continuum robots (TDCRs), the class of CR explored in this work, will first be presented. What follows is an overview of magnetic interactions as they pertain to the outlined research problem. Finally, a discussion of vibrations in TDCRs, vibration suppression, and similar challenges in other cable driven systems is given, informing the chosen direction of future work.

2.1 Tendon Driven Continuum Robots

TDCRs are a prominent class of continuum robots characterized by a flexible backbone actuated by cables or tendons routed along the structure. TDCRs achieve motion through the continuous deformation of their compliant backbone, driven by forces exerted through the movement of their tendons [10]. The mechanical structure of a TDCR typically consists of a central backbone, often made of super-elastic materials such as Nitinol which provide structural integrity while allowing for bending. Tendons are guided along this backbone

through spacer disks or integrated channels [10]. Depending on how tendons are routed and terminated, a TDCR is said to have a certain number of segments, which determines the type and variety of backbone shapes it can experience. To allow for bending in all directions, TDCRs are typically designed with 3 or 4 tendons terminated at the end of each segment, spaced equidistant along the robot’s circumference [11].

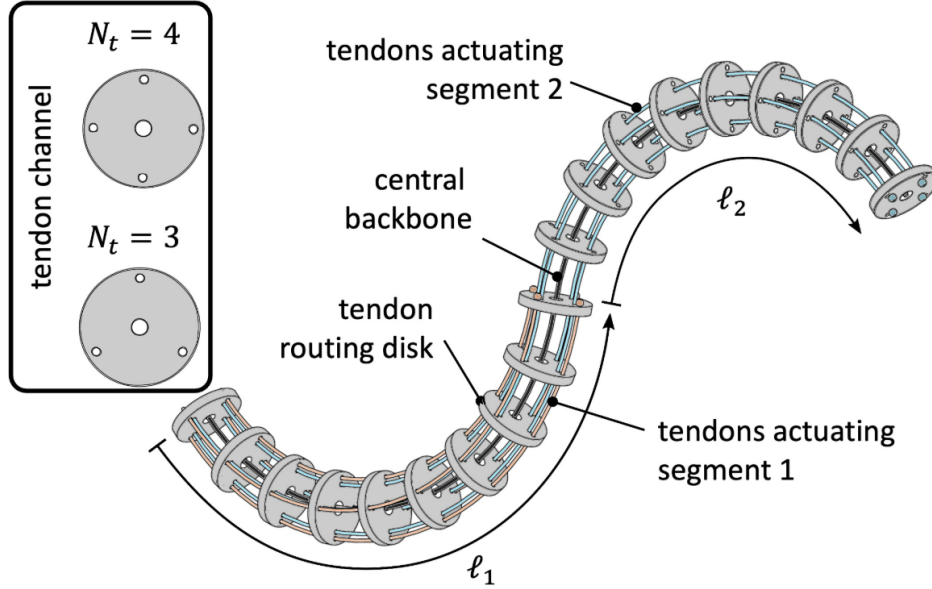


Figure 1: Diagram of a Typical TDCR [11]

Actuation of TDCRs is extrinsic. Motors located in a base module pull or release individual tendons to generate moments that bend each segment [10]. Remote actuation keeps the main robot body lightweight and compact, which is desirable for minimally invasive surgical applications [1]. Moreover, this simple and well understood actuation framework is what makes TDCRs among the most popular designs; they are relatively simple to manufacture and control compared to other CR types [10]. There are however still a number of challenges with TDCRs. These include tendon friction complicating modeling and control, the need for proper tendon tensioning systems, and increasing tendon count with each segment making miniaturization difficult [10, 12]. Finally, the inherent compliance that grants TDCRs their dexterity also introduces challenges in maintaining stability and precision under external loads, necessitating advanced methods for stiffness modulation and control [13].

2.1.1 Stiffening Approaches

There have been a number of methods proposed for modulating the stiffness of CRs allowing them to be compliant and soft when navigating but rigid when they are required to withstand external forces, for example during manipulation. A recent review by Fan et al classify these approaches into four main categories: jamming mechanisms, thermally responsive materials, mechanical structures, and antagonism [2].

Jamming Mechanisms

These techniques rely on the friction between particles, fibers, or layers of material to change stiffness [2]. Granular jamming involves vacuuming air from a membrane filled with particles (which are normally free to slide past each other) to lock them in place, drastically increasing rigidity. Layer jamming operates on a similar principle using stacked sheets. While effective, these systems tend to add considerable size and weight to a design, and require additional pneumatic tethers [2]. For these reasons, they are best suited for larger scale CRs.

Thermally Responsive Materials

Materials such as Shape Memory Alloys and Low Melting Point Alloys change their stiffness in response to temperature changes [2]. These materials can be embedded in CRs for a wide range of stiffness, however they are often limited by slow response times related to the heating and cooling cycles of the materials. In addition, they add additional thermal management concerns when placed within biological environments [2].

Mechanical Structures

Mechanical structures encompasses specific design elements which mechanically lock or restrict motion. In the case of CRs tailored to medical applications, size constraints make the miniaturization and actuation of such mechanisms challenging [2]. Robot joining most closely falls under this category, however due to the fact that stiffness increases are achieved through bracing and not internal mechanisms restricting motion within a single CR, this approach is less limited by volume constraints.

Antagonism

Antagonism uses opposing forces to increase the internal tension of the robot, thereby increasing its stiffness. In TDCRs, this is commonly achieved by pulling antagonistic tendons simultaneously [2]. While simple to implement using existing actuation hardware, stiffness gains are often coupled with the robot's configuration and limited by tendon strength [2, 13].

2.1.2 Constant Curvature and Piecewise Constant Curvature Modeling

Continuum robots fundamentally differ from traditional rigid-link robots because their continuous, flexible backbones theoretically possess infinite degrees of freedom [14]. Developing an exact analytical model that accounts for the spatial position of every infinitesimally small point along the robot’s length is highly complex and computationally prohibitive [10]. To make continuum kinematics tractable, especially for real-time control applications, the constant curvature (CC) approximation is the most widely adopted kinematic framework in the field [14].

The Constant Curvature Assumption

The CC approach assumes that when subjected to internal actuation forces (such as the tension applied by routing cables in a tendon-driven continuum robot), a continuum segment bends uniformly, forming a perfect circular arc. Under this geometric assumption, the spatial configuration of any individual segment can be entirely defined by a set of three independent arc parameters [14]:

- κ : The curvature of the arc, defined as the inverse of the radius of curvature ($1/r$).
- ϕ : The angle of the bending plane relative to the base coordinate frame’s x-axis.
- l : The total arc length of the segment.

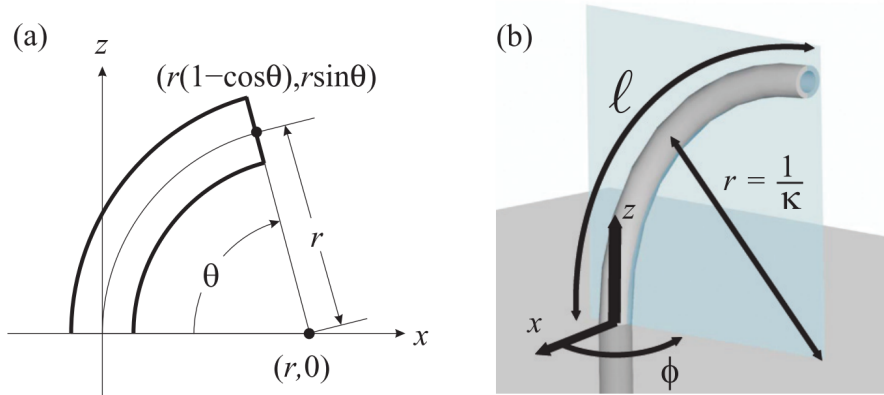


Figure 2: Diagram Depicting CC Bending [14]

While these arc parameters define the segment’s geometry in configuration space, they must be related back to the physical actuator inputs, namely the changes in length of the driving tendons in a TDCR. In a typical TDCR, tendons are routed through spacer disks at fixed

radial offsets from the central backbone. The mapping from actuator space (tendon displacements) to configuration space assumes that the tendons form continuous curves parallel to the backbone's constant curvature arc. Based on this geometry, direct algebraic relationships connect the change in length of each individual tendon to the resulting backbone curvature κ , bending plane angle ϕ , and segment length l [14, 10]. For a segment with tendons routed at a constant radial offset r , the length l_i of the i -th tendon is geometrically defined as:

$$l_i = l(1 - \kappa r \cos(\phi - \theta_i)) \quad (1)$$

where θ_i represents the fixed angular position of the i -th tendon around the central backbone. Using these arc parameters, the forward kinematics mapping the configuration space to the task space can be derived geometrically [14]. The Cartesian coordinates $\vec{p}$ of the segment's distal end relative to its base frame are given by:

$$\vec{p} = \begin{bmatrix} \frac{1 - \cos(\kappa l)}{\kappa} \cos(\phi) \\ \frac{1 - \cos(\kappa l)}{\kappa} \sin(\phi) \\ \frac{\sin(\kappa l)}{\kappa} \end{bmatrix} \quad (2)$$

In the singular case where the robot is perfectly straight (as $\kappa \rightarrow 0$), the tip position naturally resolves to $[0, 0, l]^T$.

Piecewise Constant Curvature (PCC) Modeling

For continuum robots composed of multiple independently actuated sections, the single CC model is logically extended into the Piecewise Constant Curvature (PCC) model. The PCC framework treats the robot as a serial chain of mutually tangent circular arcs [14]. The overall kinematic chain is constructed by assigning a local coordinate frame to the distal end of each segment. By utilizing homogeneous transformation matrices, the base of a subsequent segment is mapped to the tip of the previous one [10]. This concatenation allows for rapid computation of the robot's total forward kinematics, efficiently mapping actuator inputs (tendon displacements) to segment arc parameters, and subsequently to the final end-effector pose.

Advantages and Limitations

The primary advantage of the CC and PCC models lies in their mathematical simplicity and computational efficiency. Because they yield closed-form analytical solutions, they are highly suitable for real-time feedback control. This makes the PCC model exceptionally well-suited for high-speed dynamic tasks, such as the IMU-driven snapping compensation loop

utilized in this work. However, the CC assumption remains a pure geometric approximation. Literature evaluating CR modeling performance notes that real-world TDCRs inevitably deviate from true constant curvature [10]. Unmodeled physical effects such as external tip loads, gravitational sagging, non-uniform backbone properties, and tendon friction along the spacer disks cause the local curvature to vary along the length of a single segment [10]. While highly dynamic or heavy load-bearing scenarios may require advanced mechanics-based models (such as Cosserat rod theory) to capture these nonlinearities, the PCC model remains a robust and effective foundational approximation for unloaded trajectories and control schemes where computational agility is paramount [10, 14].

2.1.3 Sensing

Accurate control of TDCRs requires robust sensing strategies to estimate the robot’s complex, non-linear shape and detect interactions with the environment [15]. Sensing methodologies in continuum robotics are generally divided into shape perception and environment perception, though in the case of this work, shape perception is the focus. Due to the infinite degrees of freedom of continuum bodies, shape sensing is challenging. Optical sensors, particularly Fiber Bragg Grating sensors, are widely considered the state-of-the-art due to their high accuracy, immunity to electromagnetic interference, and small footprint [16]. However, they are costly and require complex interrogators. Alternative approaches include electromagnetic (EM) tracking, which offers direct pose measurement but is susceptible to magnetic interference, and electric sensors (resistive/capacitive) that measure strain directly on the robot body [16]. Recent work has demonstrated the efficacy of using inertial measurement units (IMUs) at a CR’s tip for closed-loop control. IMUs provide precise orientation and acceleration feedback which can be used to steer the distal end of a robot and compensate for unmodeled elasticities or external disturbances [17]. Boundary measurement approaches such as tendon tension sensors and force torque sensors placed at the base of CRs are also desirable since they do not need to be integrated in the main body of the CR and as such are exempt from many constraints. These measurements, when coupled with a system/robot model, have shown to be effective for state estimation [15]. However, in cases where quick feedback is required such as in disturbance rejection, reconstructing a robot state through the measurement of these boundary conditions can be computationally expensive and diminish agility of a control loop. Such measurements are also more difficult to obtain reliably, being subject to effects such as tendon friction [15].

2.2 Vibrations in TDCRs & Similar Cable Driven Systems

The structural dynamics of a TDCR are often modeled similarly to a cantilever beam, where the backbone exhibits continuous deformation [18]. Because TDCRs rely on flexibility for navigation, they are prone to low-frequency vibrations that take significant time to decay. While high tendon tension can stiffen the structure and alter natural frequencies to avoid resonance, it does not inherently dissipate energy efficiently without active control strategies. Furthermore, simultaneous regulation of position and stiffness remains a complex challenge in underactuated continuum systems, as stiffness adjustments often couple with configuration changes [13].

The vibration challenges faced by TDCRs share similarities with other cable-driven systems, particularly overhead cranes and Cable-Driven Parallel Robots (CDPRs). In these systems, the flexibility and low mass of their cables lead to the similar issues of payload swing and residual vibration [19, 20]. The literature on these cable-driven systems seems to highlight two primary categories of vibration suppression relevant to vibration in TDCRs: input shaping and active feedback control.

Input shaping is a widely adopted feed-forward technique in crane control, designed to limit residual vibrations induced by the system’s own motion. This method involves convolving an operator’s command signal with a sequence of impulses such as Zero Vibration (ZV) or Zero Vibration and Derivative (ZVD) shapers to create a command profile that cancels the system’s oscillatory modes [19]. It’s been demonstrated that optimization-based input shaping can effectively suppress sway in cranes even with system parameter uncertainties [16]. The limitation of input shaping is its fundamentally open-loop nature; it is effective at preventing vibrations expected to be caused by commanded movements, but less so at rejecting unmodeled, abrupt external disturbances [19]. The case of magnetic snapping in TDCR and all its unpredictable variations falls in the latter category, with the disturbance being a magnetic force that is suddenly removed from the system. As such, these methods find limited use in this context.

Active feedback control strategies which modulate system compliance in real-time are critical for disturbance rejection. In the context of CDPRs it’s been demonstrated that integral force feedback (IFF) through IMU measurements on a CDPR end effector can successfully mitigate vibrations by using tension modulation to artificially increase the damping ratio of the system, all without requiring an exact model of the complex cable dynamics [21]. For continuum robots specifically, Yi et al. propose a passivity-based controller that utilizes the

antagonistic nature of tendons to actively regulate stiffness and damp out transient energy [13]. Additionally, setpoint controllers that incorporate specific vibration damping terms have been shown to significantly improve settling times in continuum backbones [22]. These active damping approaches suggest that by rapidly modulating tendon tension in response to the magnetic unjoining event, the transient vibrations in a TDCR can be actively suppressed, stabilizing the system quickly.

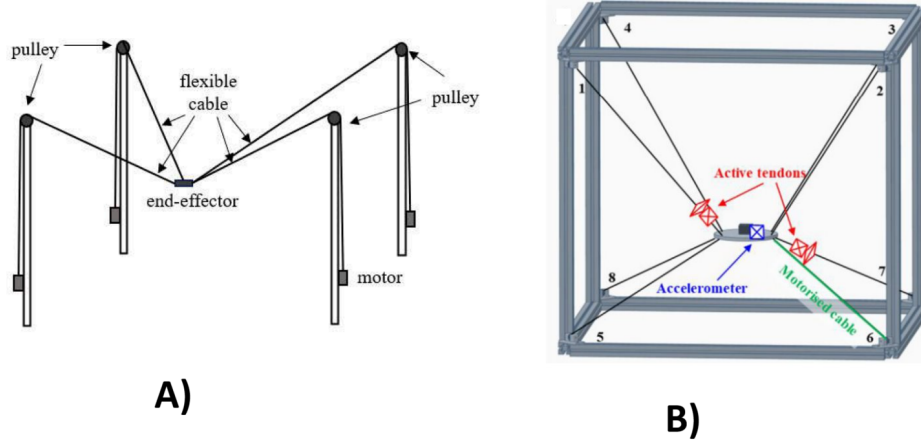


Figure 3: Examples of CDPRs [21, 20]

2.3 IMUs in Vibration Suppression Systems

An Inertial Measurement Unit (IMU) is an electronic device that measures and reports a body’s specific force, angular rate, and sometimes orientation, using a combination of accelerometers, gyroscopes, and occasionally magnetometers. Modern IMUs are predominantly built using Micro-Electro-Mechanical Systems (MEMS) technology. This miniaturization has drastically reduced their cost, size, and power consumption while maintaining high performance, allowing them to be embedded directly into the end-effectors of robotic systems, flexible appendages, and other highly dynamic structures [16].

In the context of vibration suppression, systems generally fall into two categories: passive damping, which relies on materials or mechanical dampers to absorb energy, and active vibration control (AVC), which utilizes sensors, controllers, and actuators to dynamically cancel out oscillations in real-time [20]. Within AVC frameworks, IMUs serve as a critical feedback mechanism. While traditional boundary sensors, such as base-mounted force-torque sensors or motor encoders, struggle to accurately estimate the high-frequency dynamic states of flexible distal ends, a tip-mounted IMU provides direct, localized measurements of the

structural vibrations [15]. Specifically, the embedded accelerometers capture the linear accelerations associated with the vibratory sway, while the gyroscopes capture the angular velocities crucial for detecting rotational oscillations and torsional vibrations. By feeding these inertial measurements into a closed-loop control system, actuators can be commanded to apply counter-forces or structural tensioning to actively dissipate the kinetic energy of the system [17].

The effectiveness of such an IMU-based vibration suppression system is heavily dictated by the hardware specifications of the sensor. For a controller to successfully damp out transient vibrations, the IMU must possess a high output data rate (ODR) and wide bandwidth. According to the Nyquist-Shannon sampling theorem, the sensor must sample at a rate at least twice the highest frequency of the structural vibration, though practical closed-loop control often requires sampling an order of magnitude higher to ensure phase fidelity [9]. Modern MEMS IMUs can achieve ODRs in the kilohertz range, making them highly capable of capturing fast transient events such as impact shocks. Furthermore, because vibrations can have very small amplitudes as they begin to damp out, the sensor must have a low noise density to ensure the signal can be distinguished from electrical noise without relying on heavy, phase-lagging low-pass filters. Finally, low latency is critical; delayed feedback can inadvertently add energy to the system and exacerbate the vibration, which makes hardware-level communication protocols preferred over slower alternatives to minimize transmission delays [17].

A primary challenge in using IMUs for kinematic control is the inherent noise and bias drift present in MEMS sensors. While acceleration data is highly responsive, utilizing it directly to calculate displacement requires double integration, which causes low-frequency noise and sensor bias to accumulate rapidly into massive positional drift [?]. To combat this, vibration suppression controllers rarely rely on absolute position estimates derived purely from an IMU. Instead, control strategies typically utilize direct acceleration feedback, feeding raw or lightly filtered acceleration data directly into the control loop to command an opposing force proportional to the acceleration. Alternatively, controllers may employ velocity feedback for active damping by single-integrating the acceleration data and applying a high-pass filter or leaky integrator to continuously strip away accumulating drift. Applying a control force opposite to this estimated velocity artificially increases the damping ratio of the mechanical system [20]. These IMU-driven strategies are widely utilized across various flexible architectures, including Cable-Driven Parallel Robots (CDPRs), overhead cranes, and highly compliant continuum robots, where active tension modulation can rapidly stabilize

the system following a dynamic disturbance [20, 13].

3 Methods

3.1 Problem Simplification

Given the complexity of the snapping problem with its many coupled variables influencing the final phenomenon, the choice was made to simplify down to the case of a single magnetic CR constrained to planar movement interacting with a fixed set of permanent magnets. In this way, the simplest form of snapping may be analyzed and core methods to dampen vibrations can be developed.

The experimental construction decided upon consists of a TDCR with two equal length segments, each having a pair of tendons routed parallel to each other, allowing for 2 bending degrees of freedom in a plane. Embedded in each of the spacer disks adhered along the robot's nitinol backbone are axially magnetized toroidal neodymium magnets with alternating orientations so that each disk repels its neighbors. This seeks to enforce spacing between disks, as well as to create a magnetic pattern allowing flexible joining with another CR containing magnetic disks of mirrored polarities. The robot rests on a level plate which defines its plane of motion and serves to counter-act bending induced by gravity. Finally, a parallel fixed set of magnets are arranged linearly to mirror the robots configuration, with the addition of a global offset to allow for magnet alignment when the robot bends in for joining. In effect, this fixed set of magnets represent a secondary magnetic CR with extreme rigidity in its straight configuration. The magnetic CR to be constructed measures 10 mm in diameter and 140 mm in length, representing a size consistent with a TDCR tailored for surgical applications.

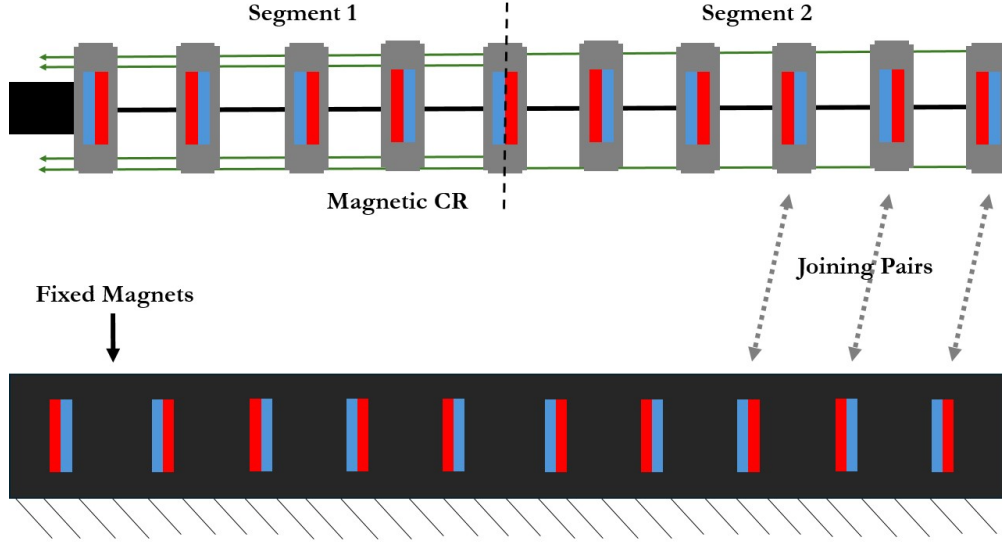


Figure 4: Diagram of Simplified Snap Testing Configuration

With this configuration, the magnetic CR can assume 2D poses to interact and join with the set of fixed magnets at it's distal end. Given the robot's cantilevered nature, a join at its distal end is that which both most increases its stiffness and is most likely to result in elastic build-up and snapping. It is for this reason that this joining configuration is targeted. The tests detailed in section 5 will consist of breaking this distal join and aiming to return to the robot's neutral (straight) position with minimal overshoot in accordance with metric I.

3.2 IMU Sensor Board Development

While IMU integrated circuits (ICs) are commonly available in package footprints as small as a few millimeters, off-the-shelf PCBs providing interfaces with these sensors are often not optimized for the compact integration required for embedding these sensors in surgically oriented CRs. As a result, integrating an IMU into this work necessitated both the selection of not only an appropriate IMU IC, but also an accompanying interfacing PCB design to allow for integration within this application's size constraints.

3.2.1 Robot Vibration Frequencies

To fully define metrics II and III, the vibrational modes induced in a test setup of the type described in section 3.1 are analyzed. Figure 5 shows the constructed setup, fabricated largely through the use of FDM 3D printing. The magnetic CR contained within is composed of a 0.025 inch diameter nitinol backbone, tendons of Dyneema fishing line, and 10 resin 3D printed spacer disks with embedded magnets as previously described. Featured as well is an

overhead platform to be used for the mounting of a camera, with an orange marker applied to the tip of the robot to facilitate tracking. Tracking is achieved through the color masking of individual camera frames and computing the pixel centroid of the resulting mask, as shown in figure 6. Finally, a 3D printed servo cage adapted from the open-source ORCA hand project is bolted to the rear of the setup, and contains the CR's tendon actuating servo motors [23].

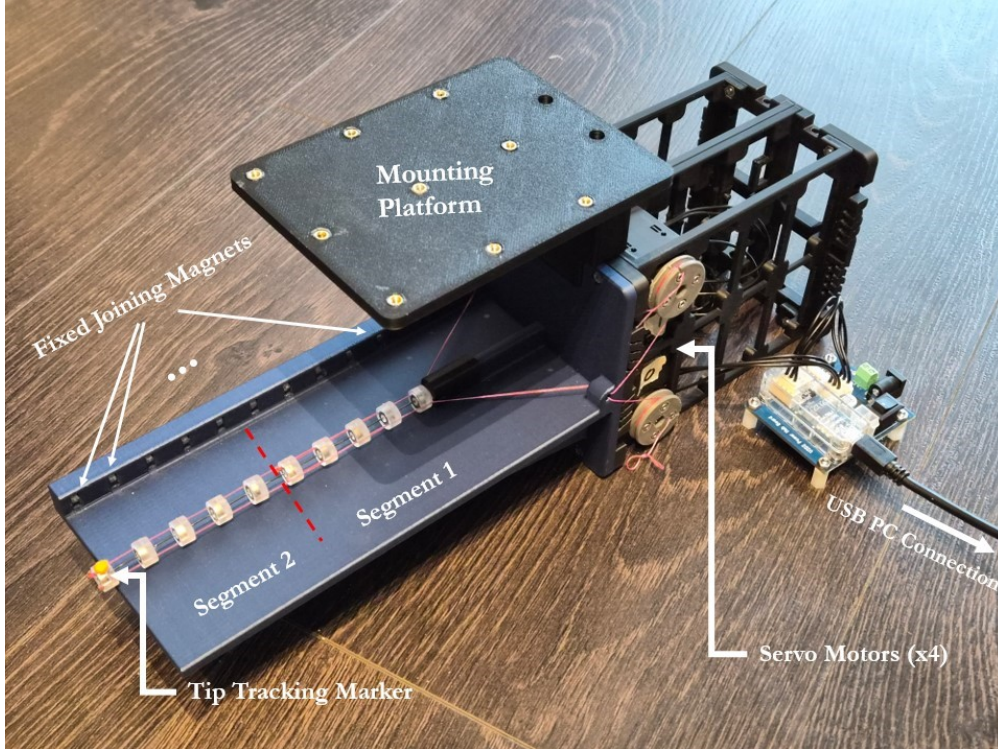


Figure 5: Planar Test Setup



Figure 6: Example Frame of Tracking Footage (Top) & Tip Position Estimate (Bottom)

To extract vibrational frequencies to inform IMU selection, a test is devised whereby all tendons are disconnected from their motors and the magnetic CR is distally unjoined manually through a knock at the joining location. This ensures free vibration of the robot structure allowing a clear sinusoidal response fit for analysis. Importantly, without the presence of any passive tendon tension as is typical in standard TDCR operation, the effective stiffness of the robot is minimized and a lower natural frequency is expected. This will require the resulting IMU requirements to be conservative in relation to the measured frequencies. Figure 7 plots the horizontal position of the robot tip over time as it unjoins and vibrates in-plane. A damped two-mode vibration is observed with a peak displacement from the joining location of approximately 500 pixels or 58mm, calculated through computing a rough pixel/mm ratio which neglects lens distortion effects. Fitting the data to a curve of the form in Eq. (1), the frequencies and damping factor presented in table 1 are found.

$$x(t) = e^{-\gamma_1(t-t_0)} [a_1 \sin(\omega_1(t-t_0) + \phi_1)] + e^{-\gamma_2(t-t_0)} [a_2 \sin(\omega_2(t-t_0) + \phi_2)] + x_0 \quad (3)$$

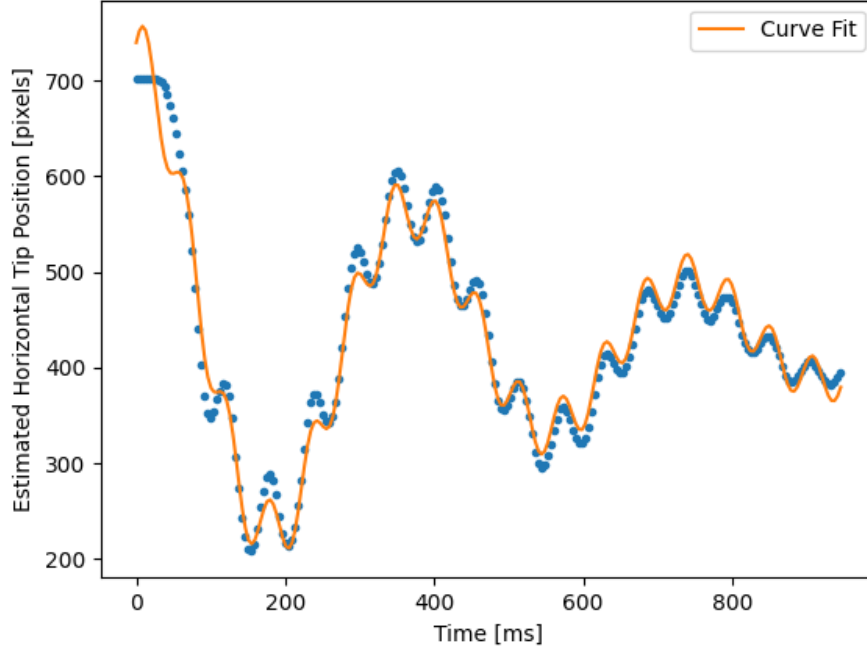


Figure 7: Fitted Curve Against Free Vibration Data

Table 1: Results of Curve Fit

Primary Frequency (ω_1)	Secondary Frequency (ω_2)	Primary Damping Factor (γ_1)
2.7 Hz	17.8 Hz	2.09×10^{-3}

The results in table 1 show a primary vibration frequency of 2.7Hz and corresponding period of 370 ms, with a damping factor which results in a 2% settling time of 1.91 seconds. Interestingly, the free vibration test allows the development of a second order vibration at 17.8 Hz, something which is unlikely to be excited in tendon driven operation. Nonetheless, this second order frequency is chosen as the conservative upper bound of frequencies to be captured by the selected IMU for this application. This fully defines the frequency requirements for metrics II & III, now defined to be at least **178 Hz**

3.2.2 IMU Selection

Given the size, frequency, and quality requirements of this application, the chosen IMU for integration is the LSM6DSV16X high performance 6-axis IMU by STMicroelectronics. It's relevant specifications and relation to the application's requirements are tabulated below.

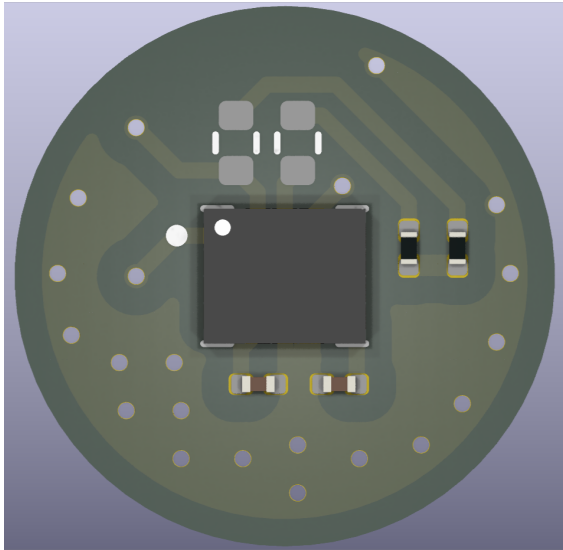
Table 2: LSM6DSV16X IMU Specifications [24] vs. Application Requirements

Feature	Specification	Requirement Relevance
Output Data Rate (ODR)	Adjustable up to 7.68 kHz	Provides ample overhead for data transmission and processing delays to achieve the minimum 178 Hz effective data rate requirement.
Acceleration Measurement Range & Sensitivity	Adjustable between $\pm$ 2-16 G	Provides flexible and appropriate measurement capabilities for capturing sudden snapping events.
Gyroscope Measurement Range & Sensitivity	Adjustable between 125-4000 degrees per second (dps)	Provides flexible and appropriate measurement capabilities for capturing rotational movements associated with snapping.
Accelerometer Noise Density	60 μ G/ $\sqrt{\text{Hz}}$ (High-Perf. Mode)	Ensures a high signal-to-noise ratio when reading minor acceleration changes during a snapping event.
Gyroscope Noise Density	2.8 mdps/ $\sqrt{\text{Hz}}$ (High-Perf. Mode)	Low noise profile provides stable rotation data with minimal drift advantageous for potential control methods utilizing tip orientation.
Package Size	2.5 x 3 x 0.83 mm	Allows for integration with a compact PCB fitting within the 10 mm diameter robot tip or other CRs of surgical size.
Current Draw	0.65 mA (High-Perf. Mode)	Low current draw allows for ultra-fine power & signal wires which can be comfortably routed down internal channels in a CR of surgical size.

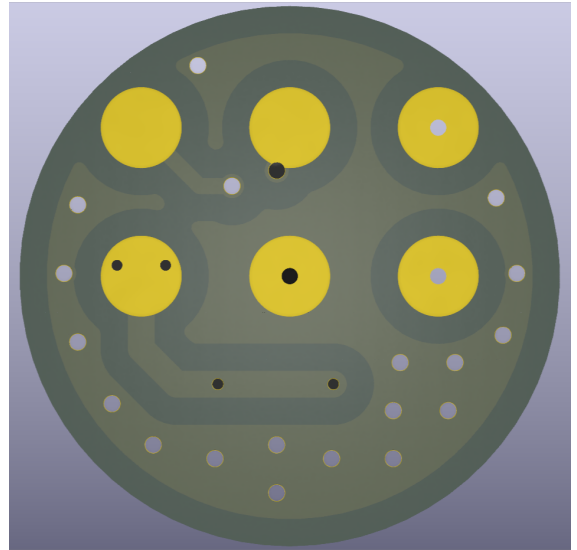
Overall, the features highlighted in table 2 which suggest high compatibility for the task at hand, matching the manufacturer’s claims for suitability in applications of vibration monitoring and compensation [24]. In addition to its strong features, this particular IMU is chosen for its commonness. This provides the practical advantages of market availability, low cost (4.42 CAD unit cost as of April 2026), and the existence of open-source firmware libraries accelerating development.

3.2.3 PCB Design

The resulting PCB designed around the selected IMU is a 2 layer construction, with the top layer containing all components, and the bottom layer being reserved for contact pads for the connection of signal and power wires to be routed down the robot. Aside from the IMU IC, the board contains the sensor’s necessary decoupling capacitors, and the addition of pull-up resistors separated by open solder bridges. The latter is a feature of experimental convenience, providing the flexibility of utilizing either Inter-Integrated Circuit (I2C) or Serial Peripheral Interface (SPI) protocols for communication with the sensor. Completing the solder bridges allows I2C communication, which while having significantly having lower bandwidth than SPI, has the advantages of requiring less wires to be routed through the robot (4 vs. 6) and being more widely compatible with host devices. The trade-off is that the additional resistors and bridge points cause board diameter to be significantly larger than it could otherwise be. The assembled board has a diameter of 10 mm and a maximum height of 2.43 mm. This is sufficient for the CR used in this work, but applications warranting a smaller size would benefit from a future design iteration removing the I2C compatibility. It should be noted that the LSM6DSV16X also supports the I3C protocol, the evolution of I2C which does not require pull-up resistors, has higher bandwidth, and maintains the light wire count of I2C [24, 25]. This would likely be the ideal protocol choice for such a future iteration. The exact circuit diagram and layout of this PCB can be found in appendix B.



(a) Top View

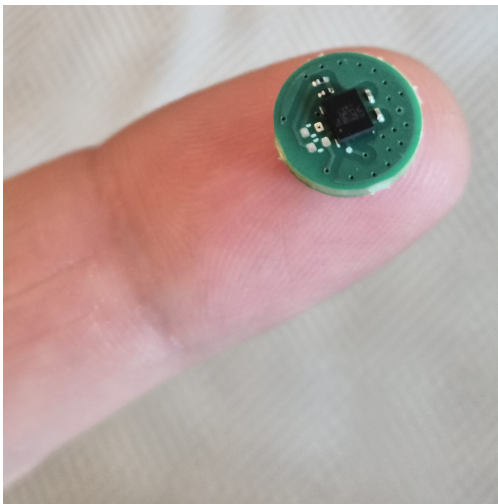


(b) Bottom View

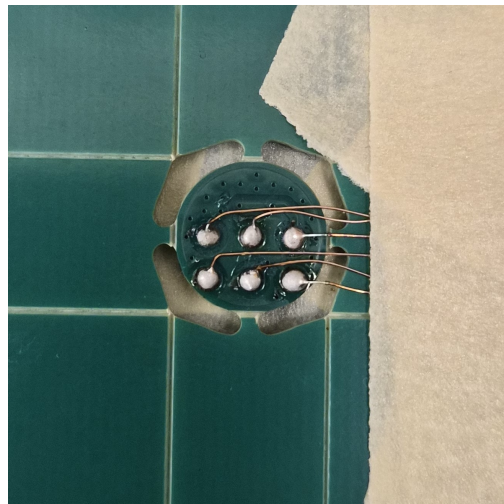
Figure 8: Software Rendering of Sensor Board Design

3.3 Final Experimental Setup & IMU Integration

A finalized experimental setup integrates the designed and manufactured IMU carrier board with the existing with the planar test setup shown in figure 5. Enamelled copper wires 0.2 mm in diameter are soldered to the rear pads of the board before being adhered to the tip of the CR, with the sensor’s X-axis pointing away from the wall of joining magnets. A dedicated microcontroller board is used to interface with the sensor over SPI for the highest possible data rate. While possible for the sensor wires to be routed down the length of the robot, for testing convenience they are exposed externally and directly connected to the microcontroller as shown in figure 11. All sensors and actuators are connected in parallel to a central computer over USB, as shown in figure 10. The exact electronic hardware used in the setup can be found in appendix B. The central computer to which the testing setup is connected gathers and processes information from the motors, camera, and IMU, in parallel. This allows for speed and accurate timing in both data collection and compensation efforts. The final effective data rate of the IMU is found through comparing consecutive timestamps of measurements which have reached the main running process on the central computer. These reveal a data rate of just over 400 Hz, exceeding the target for metric II. It should be noted that while configured internally to sample at 7.68 kHz, due to USB bandwidth limitations, the measurements reaching the microcontroller are first smoothed and then down-sampled by a factor of 7 before being sent to the computer. The remaining loss in frequency is a result of the time required to receive, unpack, and move the incoming measurements into a memory space accessible by the computer’s main process. For further information, refer to appendix sections A.3 and B.2



(a) Sensor Board with Finger for Scale



(b) Rear Soldered Connections

Figure 9: Manufactured Sensor Board

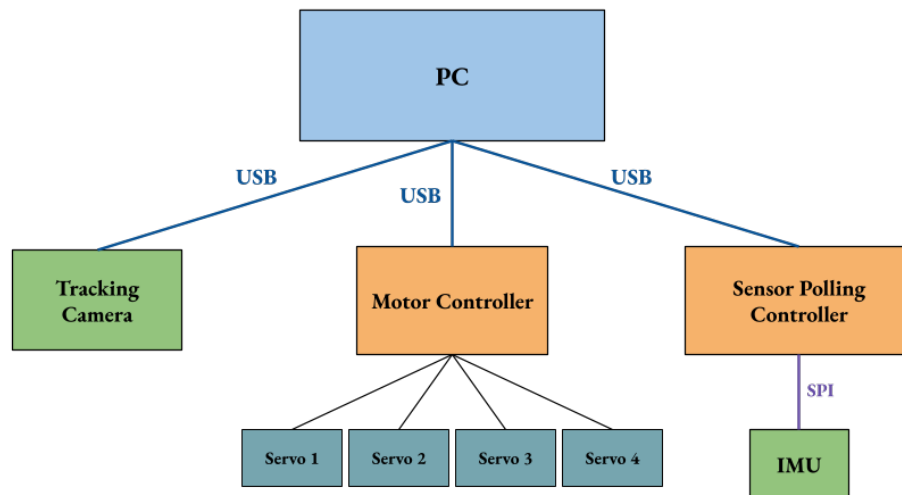


Figure 10: Experimental Setup Component Connections

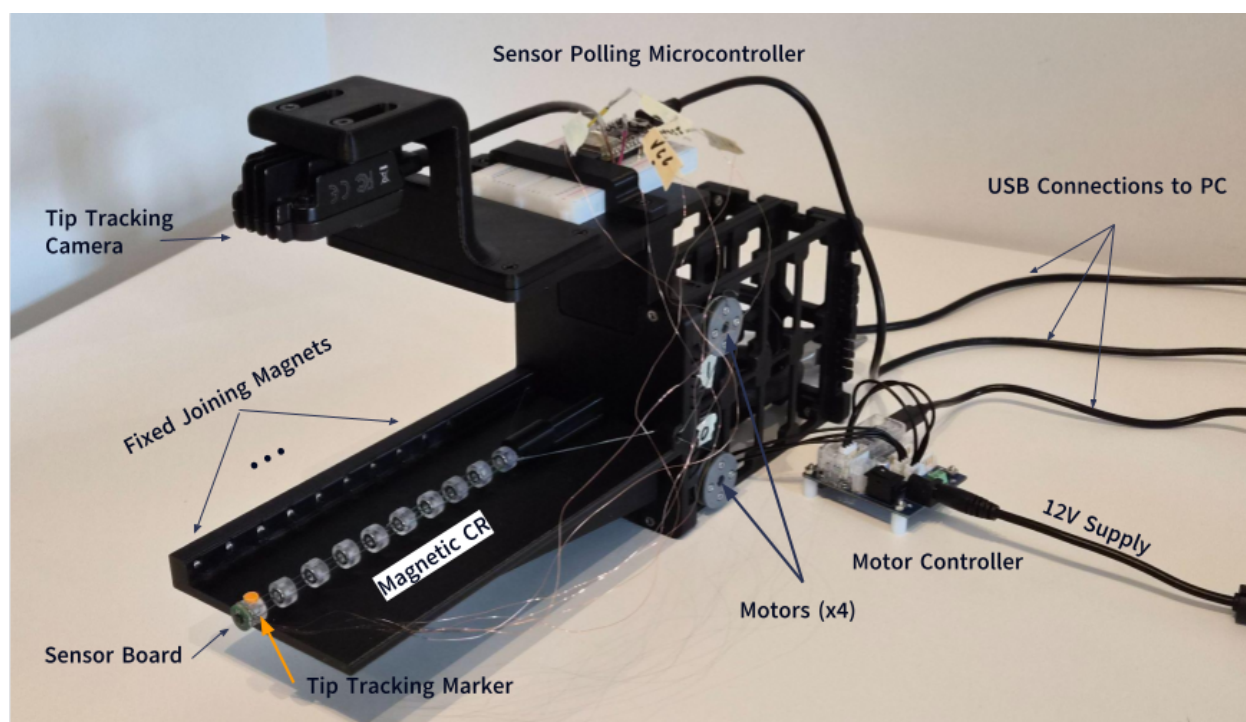


Figure 11: Final Experimental Setup

4 Testing & Results

4.1 Baseline Snapping Response

To evaluate the fundamental snapping behavior of the experimental setup without intervention, baseline tests were first conducted. These consisted of actuating the robot's proximal segment to touch the fixed magnets initiating distal joining, followed by a 1 second pause before performing opposite actuation for unjoining. The unjoining actuation is done intentionally through the proximal segment to induce a worst-case elastic build-up. Due to the attractive forces between the fixed joining magnets and the robot, commanding motor positions which would normally cause the robot to return to it's neutral configuration are insufficient to trigger unjoining. Consequently, the experimental minimum actuation necessary for unjoining was applied through gradual increases.

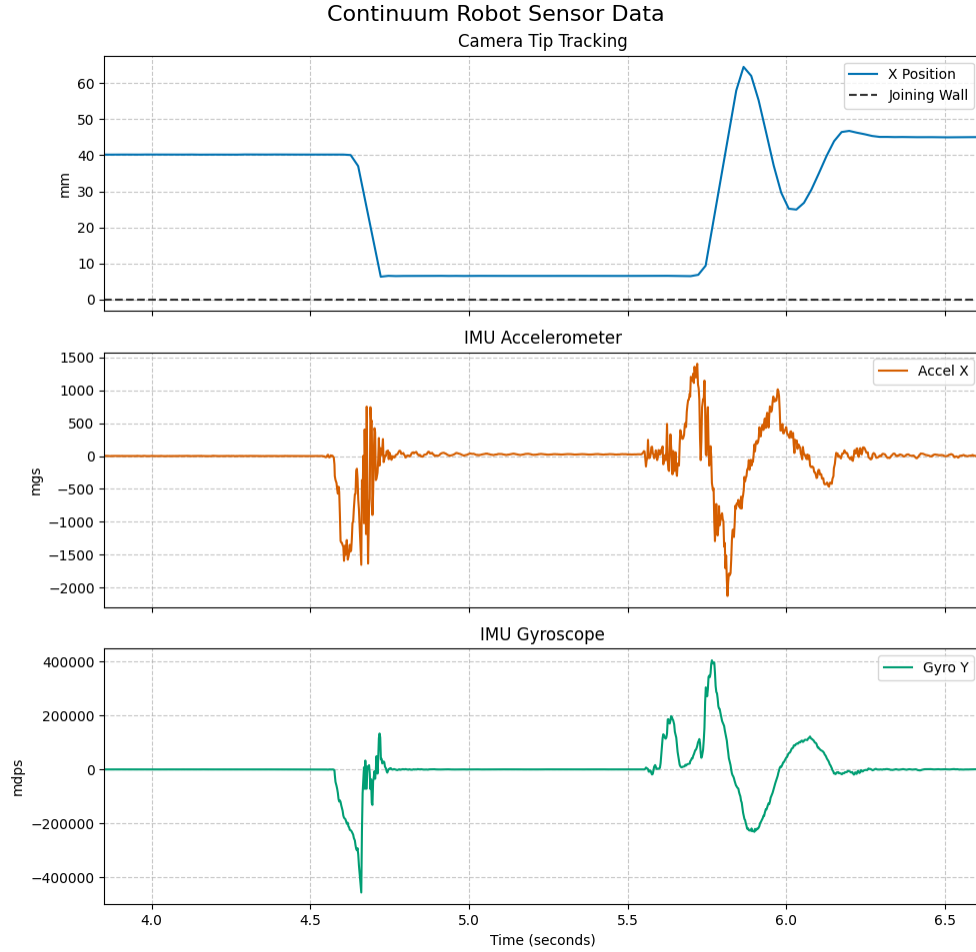


Figure 12: Baseline Snapping Response of Experimental Setup

Figure 12 shows a typical response from the baseline test procedure. Tip tracking data

obtained from the overhead camera reveals that this naive unjoining results in a snapping overshoot of 25 mm from the robot’s neutral position. The integrated IMU captured the corresponding sudden spikes in both linear acceleration (Accel X) and rotational velocity (Gyro Y) upon detachment, establishing a clear profile of an unmitigated snap event. This baseline test was performed a number of times to qualitatively check the variance in the captured IMU data, which showed very similar results on every trial. Figure 13 below shows the clearly discernible M-shaped curve of each snapping event, in accordance with metric II.

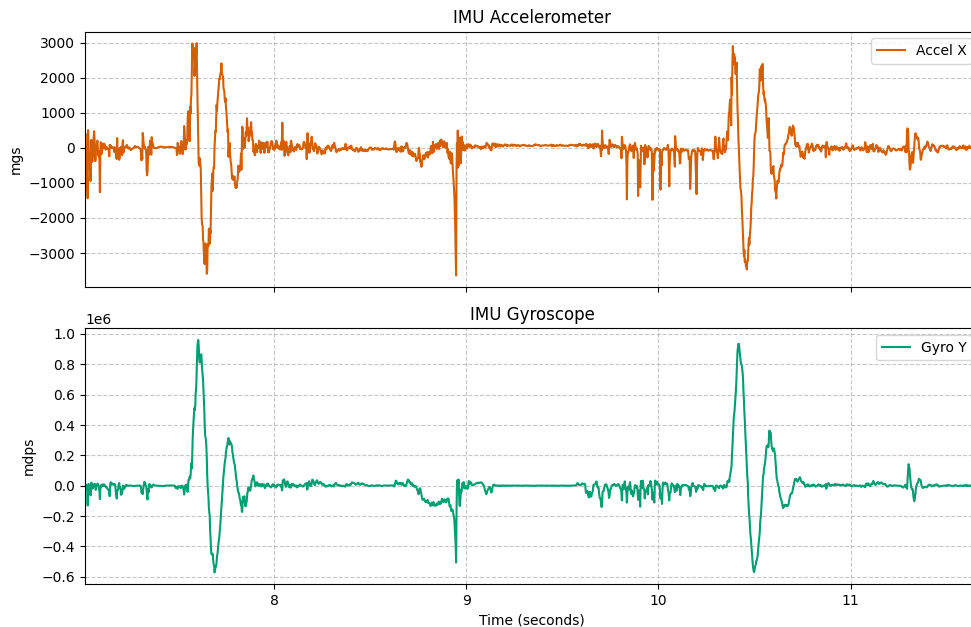


Figure 13: Captured IMU Data from 2 Consecutive Baseline Snaps

4.2 Open-loop Tendon Compensation

Consistent with objective A, investigations into the effectiveness snapping minimization as a consequence of compensatory tendon actions are first performed in isolation. In this section, two methods of open-loop compensation are discussed, uniform tensioning in response to snapping, and the execution of experimentally tailored compensation profiles.

4.2.1 Uniform Tensioning in Response to Snapping

Initial investigations into open-loop compensation explored the effects of applying naive, uniform tensioning immediately following unjoining. Intuitively, dynamically increasing the robot’s stiffness in this way was thought to be the simplest method to potentially reduce snapping. Trials were performed to determine the maximum tensioning command achievable by the robot without buckling, and this was found to be 1.2 mm of tendon retraction.

Testing the response of increasing levels of tensioning immediately after unjoining revealed no reduction or enlargement in snapping overshoot or settling time, but simply an increase in the frequency of vibration after unjoining. Figure 14 below shows the most pronounced response of these trials, with the maximum 1.2 mm retraction being performed. Ultimately, this approach proved insufficient, demonstrating the need for more sophisticated compensation techniques.

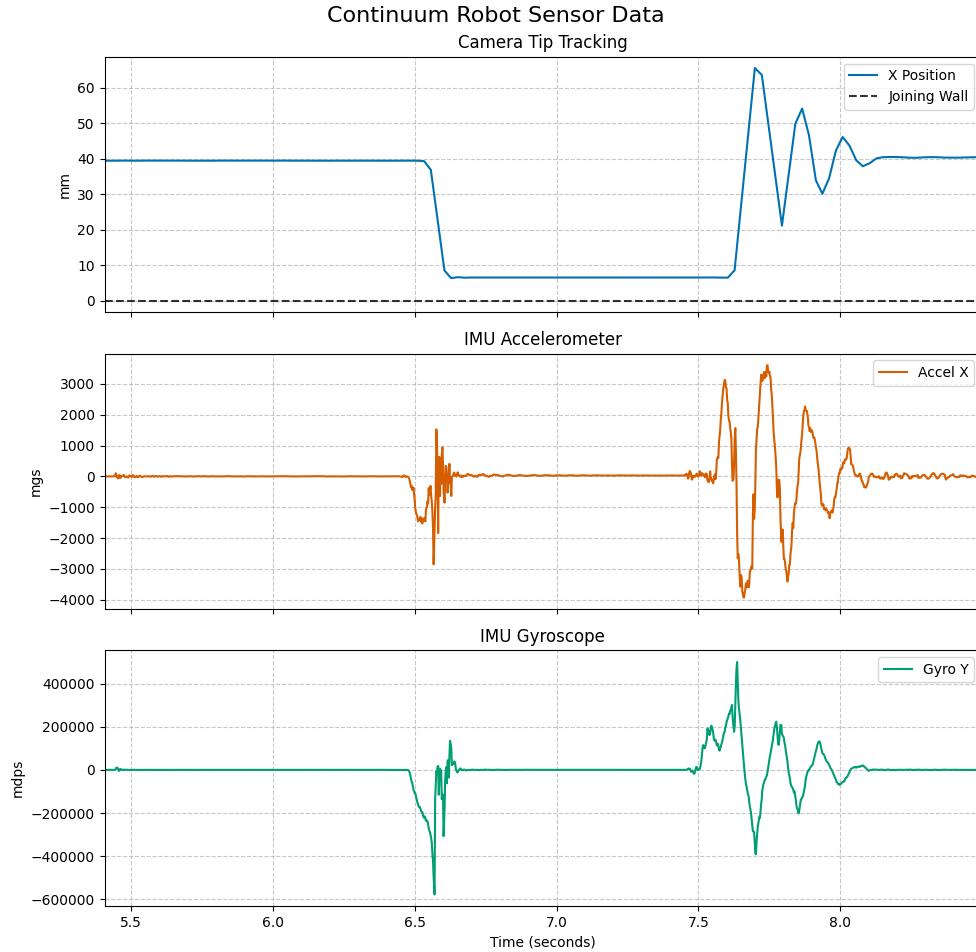


Figure 14: Effects of Robot Tensioning During Snapping

4.2.2 Execution of Pre-programmed Compensation Profiles

Building upon the limitations of naive tensioning, predetermined tendon actuation profiles were sought to be generated and executed upon unjoining to counteract snapping behavior. Through the process of trial and error, such a profile which orchestrated specific sequences of length changes across the four driving tendons prior to commanding a return to the neutral position was generated. The execution of this targeted compensation profile successfully

counteracted the elastic build-up in the system, reducing the snapping overshoot from the 25 mm baseline to merely 3 mm as shown in figure 15. This successful suppression safely satisfies the established target of metric I, staying well within the 5 mm limit.

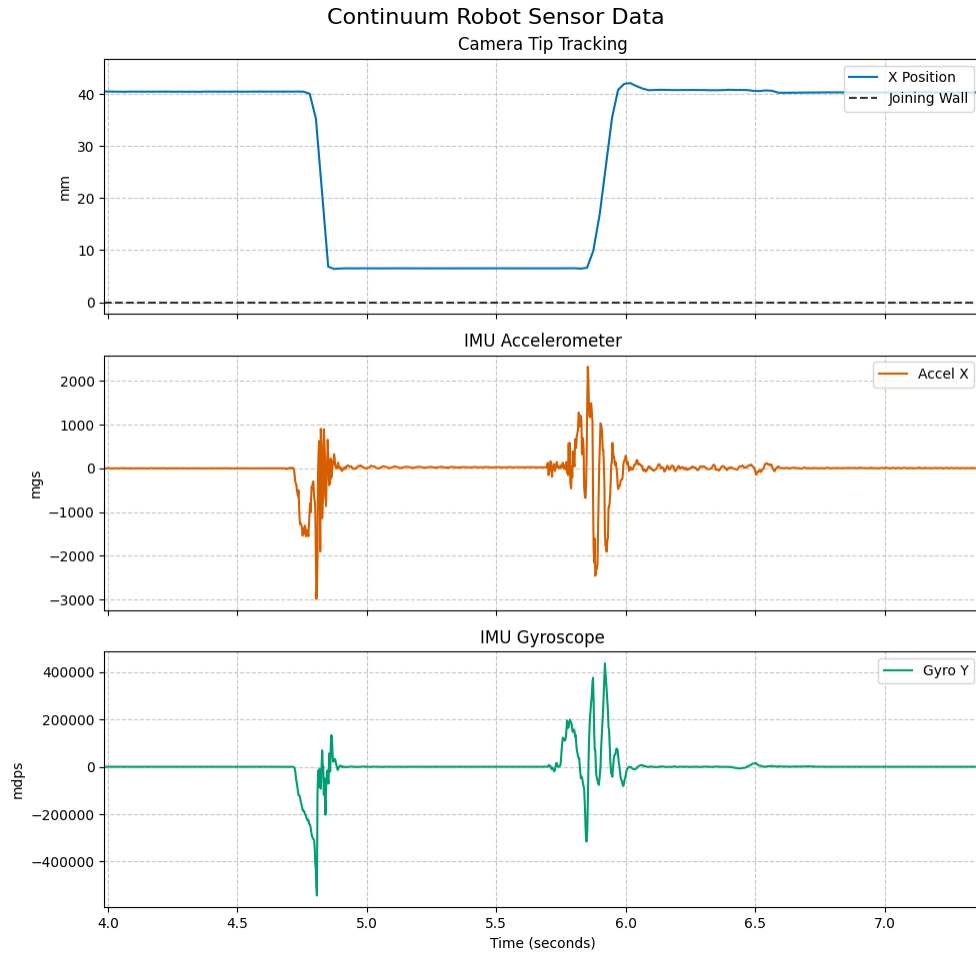


Figure 15: Robot Unjoining with Tailored Compensation Profile

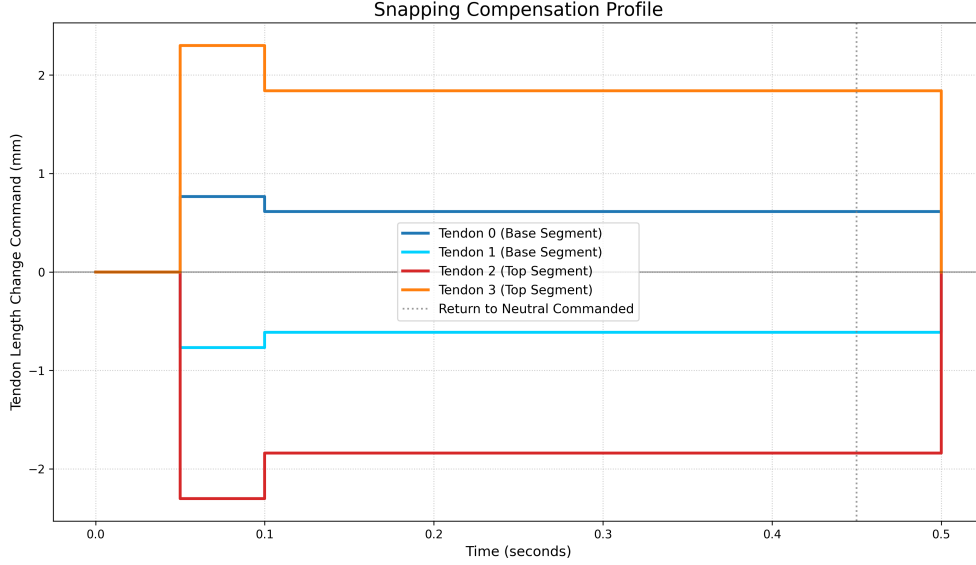


Figure 16: Effects of Robot Tensioning During Snapping

Looking at the actuation profile in figure 16 which successfully mitigated snapping, we find that a single set of tendon impulses in the opposite direction of the induced snap is sufficient in this unjoining scenario. Furthermore, we see that the distal segment is actuated much more aggressively than the base/proximal segment. This aligns with the notion that the base segment has a larger influence on tip position per unit of tendon displacement, producing bending at the robot’s midpoint which is amplified by the remaining length. This insight provides a stepping stone in developing more sophisticated compensation systems, particularly in how to assign multiple segment actuations in response to a scalar inertial measurement.

4.3 Incorporation of IMU Feedback

With the successful validation of compensatory tendon action as a method for mitigating snapping behavior in the test setup, this section aims to integrate this approach with IMU feedback. First, with IMU data acting as a trigger for predetermined compensatory behavior, and finally as a fully-fledged feedback control system.

4.3.1 IMU Triggered Compensation Profiles

As an extension to the success of the compensation profile introduced in section 4.2, the same result is sought by triggering the execution of the profile in response to IMU feedback. In these experiments, the unjoining procedure is interrupted and the predetermined compensation profile is triggered the moment that the localized acceleration exceeded 1 G. The sample

result in figure 17 shows similar behavior to the manually triggered case, proving that the IMU can serve as a reliable and fast-acting snap detection mechanism. This is particularly powerful as initial snapping acceleration or jerk vectors calculated from these measurements could be used to dynamically scale and generate compensation profiles in different directions, extending the applicability to more unjoining scenarios and 3D snapping cases.

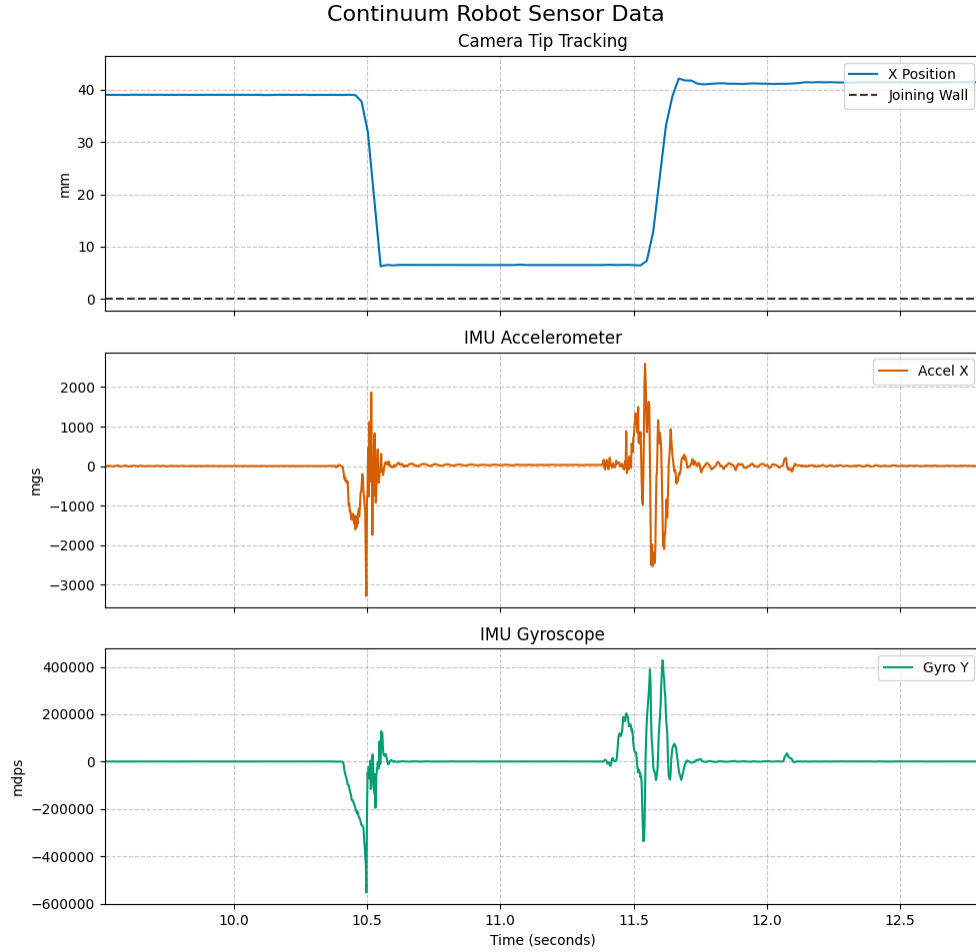


Figure 17: Robot Unjoining with IMU Triggered Compensation Profile

4.3.2 Feedback Control with IMU Data

While the tailored compensation profiles shown in previous sections effectively satisfy the outlined snapping damping requirements, relying on pre-programmed profiles is not easily generalizable across all unjoining scenarios, and especially across different TDCRs. To address this lack of universality, full active feedback control is proposed as a more effective solution.

Controller Setup

With the aim of minimizing snapping overshoot from a distal joint while returning to a desired unjoined pose, a feedback controller law which utilizes the IMU's tip acceleration measurements in addition to tendon length deviations from the desired end state is proposed. The continuum robot is modeled geometrically utilizing the piece-wise constant curvature assumption (PCC), allowing the robot's pose as result of the configuration of tendon lengths $\begin{bmatrix} l_1 & l_2 & l_3 & l_4 \end{bmatrix}^T$ to be parametrized into a curvature for each segment $\begin{bmatrix} \kappa_1 & \kappa_2 \end{bmatrix}^T$.

The horizontal coordinates $(x_1(t), x_2(t))$ along the lengths (t) of each segment are given by:

$$x_1(t) = \begin{cases} 0 & \kappa_1 = 0 \\ \frac{1 - \cos(\kappa_1 t)}{\kappa_1} & \kappa_1 \neq 0 \end{cases} \quad (4)$$

$$x_2(t) = E_x + \begin{cases} \frac{t}{\sin(a_1)} & \kappa_2 = 0 \\ \frac{\cos(a_1) - \cos(a_1 + \kappa_2 t)}{\kappa_2} & \kappa_2 \neq 0 \end{cases} \quad (5)$$

where $E_x = x_1(L_1)$ is the coordinate at the end of the first segment and $a_1 = L_1 \kappa_1$ represents the angular deflection at the end of the first segment.

As discussed in 4.2.2, the movement of the tip of the robot is disproportionately affected by the actuation of the base segment compared to the distal segment. Accordingly, we wish to determine the exact relative sensitivities of changing the curvature of each segment on the horizontal tip coordinate ($X = x_2(L_2)$) to be controlled, in an effort to replicate the behavior of the successful compensation profile. These sensitivities are represented by the partial derivatives $\frac{\partial X}{\partial \kappa_1}$ and $\frac{\partial X}{\partial \kappa_2}$.

Combining the PCC equations above, the horizontal tip position as a function of the curvatures is:

$$X(\kappa_1, \kappa_2) = \frac{1 - \cos(\kappa_1 L_1)}{\kappa_1} + \frac{\cos(\kappa_1 L_1) - \cos(\kappa_1 L_1 + \kappa_2 L_2)}{\kappa_2} \quad (6)$$

Evaluating the partial with respect to each curvature yields:

$$\frac{\partial X}{\partial \kappa_1} = \frac{\kappa_1 L_1 \sin(\kappa_1 L_1) + \cos(\kappa_1 L_1) - 1}{\kappa_1^2} + \frac{L_1(\sin(\kappa_1 L_1 + \kappa_2 L_2) - \sin(\kappa_1 L_1))}{\kappa_2} \quad (7)$$

$$\frac{\partial X}{\partial \kappa_2} = \frac{\kappa_2 L_2 \sin(\kappa_1 L_1 + \kappa_2 L_2) + \cos(\kappa_1 L_1 + \kappa_2 L_2) - \cos(\kappa_1 L_1)}{\kappa_2^2} \quad (8)$$

Note that in this case, each segment is of equal length ($L_1 = L_2 = 7\text{cm}$). To map the scalar acceleration feedback from the IMU to commanded curvatures, a segment weighting ratio based on the results above is established. Figure 18 below shows the ratio $\frac{\partial x}{\partial \kappa_1} : \frac{\partial x}{\partial \kappa_2}$ for curvatures around the neutral position up to a magnitude of 0.1 cm^{-1} .

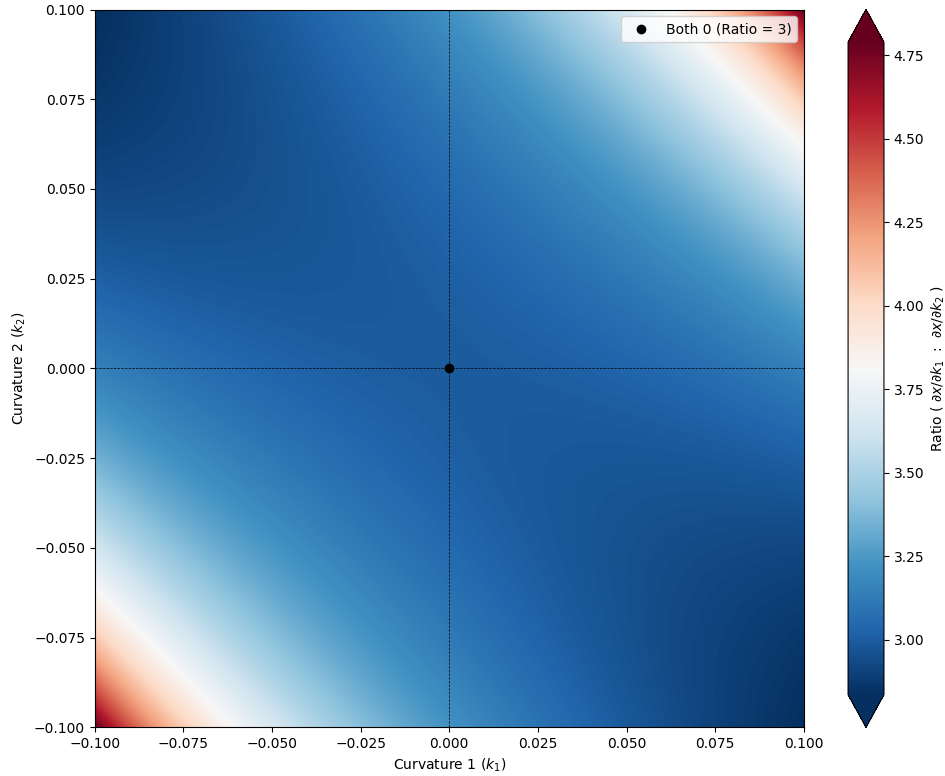


Figure 18: Relative Sensitivities of Segment Curvatures on Tip Position $\frac{\partial x}{\partial \kappa_1} : \frac{\partial x}{\partial \kappa_2}$

The results show a ratio of exactly 3:1 when both curvatures are 0, with relatively mellow variation within the plotted curvature ranges. For simplicity and as this work focuses on stabilization around the neutral position, we adopt the static normalized weightings of 0.25 and 0.75 for the proximal and distal segment curvatures, respectively.

The resulting proposed control law computes the required changes in curvature (and in turn tendon lengths) as follows:

$$u = \begin{bmatrix} \Delta\kappa_1 \\ \Delta\kappa_2 \end{bmatrix} = k_s \left(\begin{bmatrix} \kappa_1 \\ \kappa_2 \end{bmatrix} - \begin{bmatrix} \kappa_{1d} \\ \kappa_{2d} \end{bmatrix} \right) - k_a \ddot{x} \begin{bmatrix} 0.25 \\ 0.75 \end{bmatrix} \quad (9)$$

where k_s is a proportional gain acting on the difference between the steady-state curvatures that would be achieved with the current tendon lengths (κ_1, κ_2) and the final desired curvatures $(\kappa_{1d}, \kappa_{2d})$, and k_a is a gain on the IMU acceleration measurements $(\ddot{x})$. The design philosophy behind this is a simple controller which both opposes large tip accelerations and drives the robot towards it's desired end state.

Controller Evaluation and Physical Limitations

Figure 19 illustrates the system response with the controller turned off versus on with the best performing experimentally found gains. To allow for the controller to most effectively make micro-adjustments free from potential tendon slack or stretching, all tendons were tensioned to their maximum stable values as found in section 4.2 before the trials began. Without active feedback, the tip experiences the standard oscillation after unjoining, though at a higher frequency due to the higher levels of passive tension. The snapping overshoot as seen in figure 19a was measured to be 14 mm.

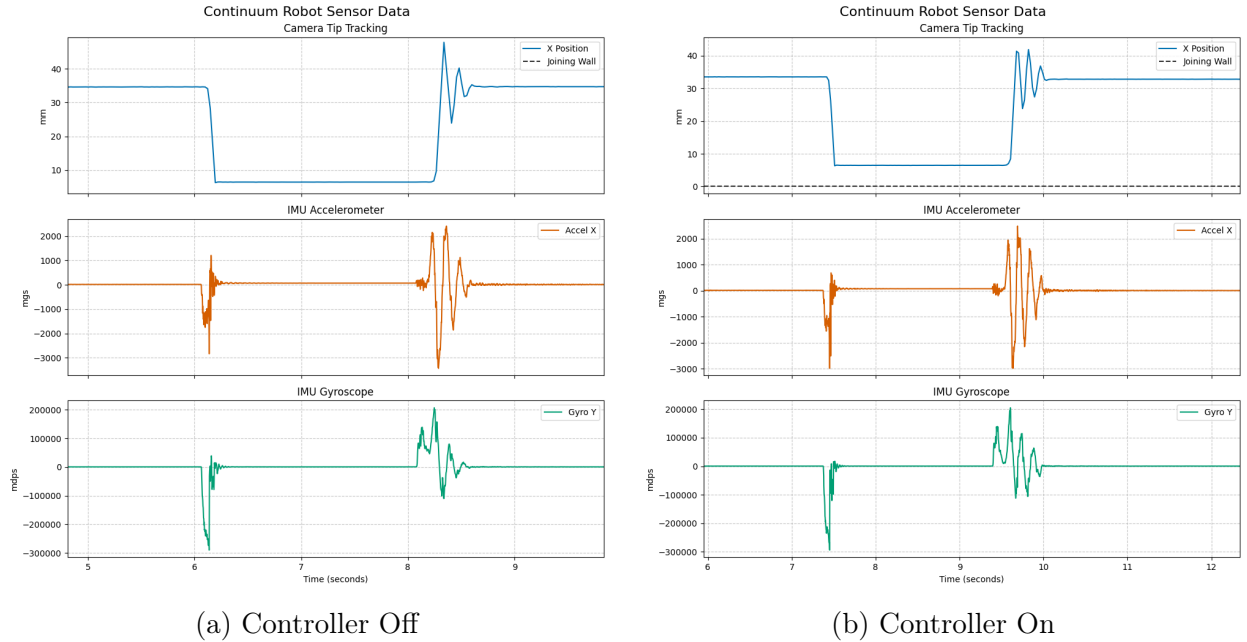


Figure 19: Sample Outputs of Feedback Controller Tests

With the controller active during unjoining, the robot's unjoining response, while lower in magnitude than the uncontrolled response does not effectively minimize snapping. The

resulting overshoot was measured to be 8 mm, with a settling time that increased from the baseline at 309ms to 404ms. The latter suggests that the tendon response lags behind the robot tip’s accelerations, causing the vibration to persist instead of diminish. This was observed more aggressively in trials with higher k_a . While the computer’s control loop sending commands to the motors was measured to be in excess of 200Hz, the true achievable command-to-action frequency of the system is likely much lower in reality, with the motor controller likely dropping/ignoring packets coming in too frequently for system’s servo motors to react.

5 Conclusion

This thesis investigated the inherent instability known as "snapping behavior" that occurs during the detachment phase of magnetically joinable tendon-driven continuum robots (TD-CRs). While magnetic joining presents a highly advantageous method for dynamically increasing the stiffness of CRs during medical force-application tasks, the elastic build-up and subsequent erratic release upon unjoining poses a significant risk to procedural safety. To address this, this work proposed and validated a novel active tendon compensation scheme driven by high-speed inertial feedback. A custom compact sensor board featuring the LSM6DSV16X inertial measurement unit was successfully designed, manufactured, and integrated into the distal tip of a planar magnetic CR. The developed electronic and software architecture achieved an effective data rate exceeding 400 Hz, surpassing the requirements for capturing the high-frequency transient dynamics of the robot.

Experimental evaluations conducted on a simplified planar test setup demonstrated the efficacy of active tendon compensation. Baseline testing revealed an unmitigated snapping overshoot of 25 mm. However, through the execution of tailored, pre-programmed tendon compensation profiles both manually triggered and triggered via IMU acceleration thresholds, this overshoot was drastically reduced to under 3 mm, safely satisfying the established 5 mm safety margin. Building upon this, the implementation of a full active feedback controller utilizing a piecewise constant curvature (PCC) model and segment sensitivity weightings looked promising for rapidly damping snapping vibrations, though final results remained lacking. Ultimately, these results provide some validation of the feasibility of utilizing tip-mounted IMUs and active tendon tensioning to stabilize magnetically joinable continuum robots.

While this work establishes a solid foundational framework for snapping mitigation, sev-

eral avenues remain for future research. First, experimental observations indicated that the standard servo motors utilized in the test setup introduced mechanical lag, occasionally struggling to match the rapid detection capabilities of the IMU. Upgrading to high-speed, direct-drive actuators would likely yield greater vibration suppression performance. Additionally, this study focused on a 2D planar simplification unjoining from a highly rigid fixed wall. Future work should expand the modeling and control efforts to move towards fully 3D snapping scenarios involving 2 independent compliant CRs. This would push magnetic CRs closer to viability in dynamic surgical environments.

- [1] J. Burgner-Kahrs, D. C. Rucker, and H. Choset, "Continuum Robots for Medical Applications: A survey," *IEEE Transactions on Robotics*, vol. 31, no. 6, pp. 1261–1280, Dec. 2015. doi:10.1109/tro.2015.2489500
- [2] Y. Fan, B. Yi, and D. Liu, "An overview of stiffening approaches for Continuum Robots," *Robotics and Computer-Integrated Manufacturing*, vol. 90, Jun. 2024. doi:10.1016/j.rcim.2024.102811
- [3] E. Walsh, private communication, Sept. 2025.
- [4] A. Lotfavar, S. Hasanzadeh, and F. Janabi-Sharifi, "Cooperative continuum robots: Concept, modeling, and Workspace Analysis," *IEEE Robotics and Automation Letters*, vol. 3, no. 1, pp. 426–433, Jan. 2017. doi:10.1109/lra.2017.2759789
- [5] C. Limpabandhu, Y. Hu, H. Ren, W. Song, and Z. T. Ho Tse, "Magnetically steerable catheters: State of the Art Review," *Proceedings of the Institution of Mechanical Engineers, Part H: Journal of Engineering in Medicine*, vol. 237, no. 3, pp. 297–308, Jan. 2023. doi:10.1177/09544119221148799
- [6] N. Wada, Y. Saikawa, H. Takeuchi, T. Takahashi, R. Nakamura, H. Kawakubo, K. Kameyama, M. Mukai, and Y. Kitagawa, "Laparoscopic Submucosal Dissection for Gastrointestinal Stromal Tumor of the Stomach: A Novel Technique for Local Excision with a Minimal Curative Margin," *Surg. Sci.*, vol. 3, no. 10, pp. 494–498, Oct. 2012, doi: 10.4236/ss.2012.310098.
- [7] D. Park et al., "Shaw scalpel use for recurrent laryngeal nerve dissection: safety parameter findings from continuous intraoperative neuromonitoring in swine models," *Gland Surg.*, vol. 9, no. 5, pp. 1363–1369, Oct. 2020, doi: 10.21037/gs-20-336.
- [8] A. Wills, D. Bates, A. Fleming, B. Ninness, and R. Moheimani, "Application of MPC to an active structure using sampling rates up to 25kHz," in *Proceedings of the 44th IEEE Conference on Decision and Control, and the European Control Conference 2005*, Seville, Spain, 2005, pp. 3176–3181.
- [9] G. F. Franklin, J. D. Powell, and M. L. Workman, *Digital Control of Dynamic Systems*, 3rd ed. Menlo Park, CA, USA: Addison-Wesley, 1997.
- [10] P. Rao, Q. Peyron, S. Lilge, and J. Burgner-Kahrs, "How to model tendon-driven continuum robots and benchmark modelling performance," *Frontiers in Robotics and AI*, vol. 7, Feb. 2021. doi:10.3389/frobt.2020.630245
- [11] J. Burgner-Kahrs, "Tendon-driven Continuum Robots," [opencontinuumrobotics.com](https://www.opencontinuumrobotics.com/101/2022/11/11/tocr-intro.html), <https://www.opencontinuumrobotics.com/101/2022/11/11/tocr-intro.html> (accessed Jan. 26, 2026).

- [12] E. Walsh, private communication, Oct. 2025.
- [13] B. Yi, Y. Fan, D. Liu, and J. Guadalupe Romero, "Simultaneous position-and-stiffness control of underactuated antagonistic tendon-driven continuum robots," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 7238–7254, 2025.
doi:10.1109/tase.2024.3462958
- [14] R. J. Webster III and B. A. Jones, "Design and Kinematic Modeling of Constant Curvature Continuum Robots: A Review," *The International Journal of Robotics Research*, vol. 29, no. 13, pp. 1661–1683, 2010, doi: 10.1177/0278364910368147.
- [15] T. Zheng and J. Burgner-Kahrs, "Estimating dynamic soft continuum robot states from boundaries," *arXiv preprint arXiv:2505.04491*, Oct. 2025.
- [8] P. J. Sincak, E. Prada, L. Miková, R. Mykhailyshyn, M. Varga, T. Merva, and I. Virgala, "Sensing of Continuum Robots: A Review," *Sensors*, vol. 24, no. 4, Art. no. 1311, Feb. 2024, doi: 10.3390/s24041311.
- [16] P. J. Sincak, E. Prada, L. Miková, R. Mykhailyshyn, M. Varga, T. Merva, and I. Virgala, "Sensing of Continuum Robots: A Review," *Sensors*, vol. 24, no. 4, Art. no. 1311, Feb. 2024, doi: 10.3390/s24041311.
- [17] M. Srivastava, R. Groff, and I. D. Walker, "Closed loop control of tendon driven continuum robots using Imus," *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2558–2563, Oct. 2023. doi:10.1109/iros55552.2023.10342138
- [18] S. V. Sotubadi, S. Li, and Y. Yang, Modeling and adaptive resonance control of cantilever beam structures using a tendon-driven continuum robot framework, Dec. 2025. doi:10.21203/rs.3.rs-8369301/v1
- [19] W. Tang, R. Ma, W. Wang, and H. Gao, "Optimization-based input-shaping swing control of overhead cranes," *Applied Sciences*, vol. 13, no. 17, p. 9637, Aug. 2023. doi:10.3390/app13179637
- [20] F. Lacaze, R. Delpoux, D. Remond, and S. Chesne, "Active vibration control of a cable-driven parallel robot using active rods," *Mechanics & Industry*, vol. 25, p. 27, 2024. doi:10.1051/meca/2024024
- [21] X. Zhao, J. Du, and K. Zhao, "Vibration suppression of redundantly controlled cable-driven parallel robots," *Robotics and Autonomous Systems*, vol. 183, p. 104838, Jan. 2025. doi:10.1016/j.robot.2024.104838
- [22] I. A. Gravagne, C. D. Rahn, and I. D. Walker, "Good Vibrations: A Vibration Damping Setpoint Controller for Continuum Robots," in *Proceedings of the 2001 IEEE International Conference on Robotics & Automation*, Seoul, Korea, May 2001, pp. 3877–3884.

- [23] C. C. Christoph et al., "ORCA: An Open-Source, Reliable, Cost-Effective, Anthropomorphic Robotic Hand for Uninterrupted Dexterous Task Learning," arXiv preprint arXiv:2504.04259, 2025. [Online]. Available: <https://arxiv.org/abs/2504.04259>
- [24] STMicroelectronics, "LSM6DSV16X: 6-axis inertial measurement unit (IMU) and AI sensor with embedded sensor fusion, Qvar for high-end applications". [Online]. Available: <https://www.st.com/resource/en/datasheet/lsm6dsv16x.pdf>
- [25] MIPI Alliance, "MIPI I3C Basic Specification", Version 1.2, Apr. 2025. [Online]. Available: <https://www.mipi.org/specifications/i3c-sensor-specification>

A IMU Sensor Board Design Information

A.1 Wiring Diagram

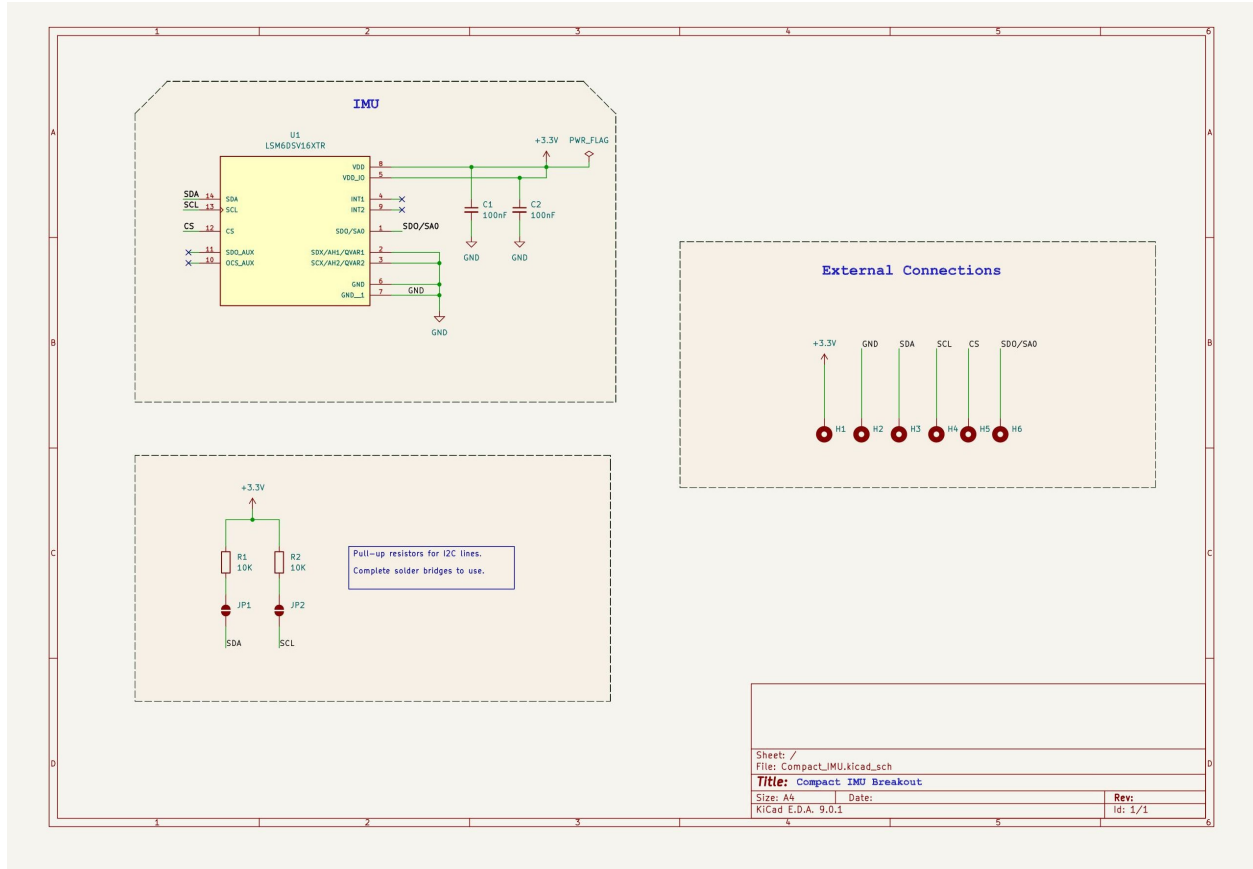


Figure 20: PCB Wiring Diagram

A.2 PCB Layout

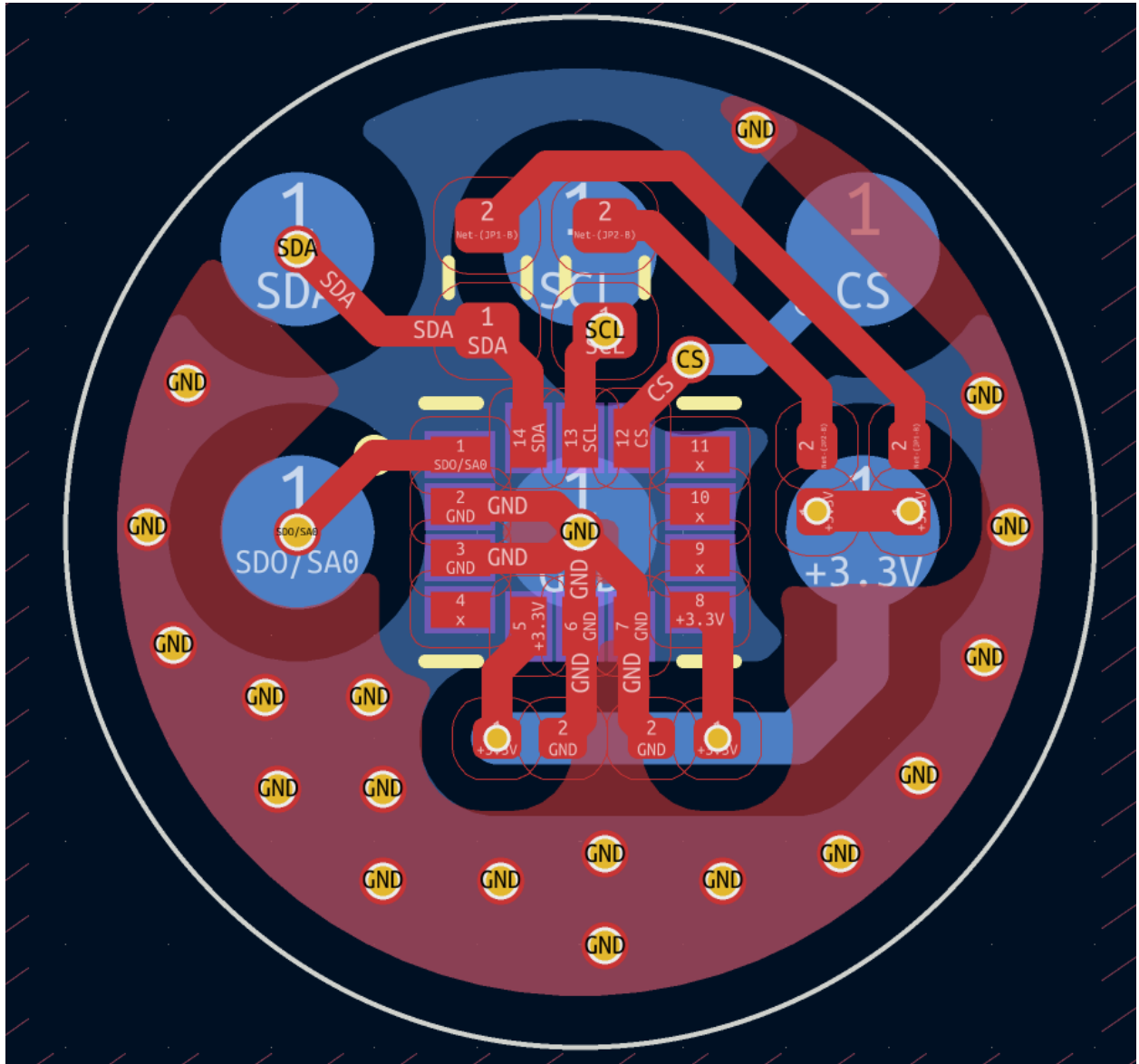


Figure 21: PCB Layout

A.3 Sensor Polling Firmware

```
1  #include <LSM6DSV16XSensor.h>
2  #include <SPI.h>
3
4  #define SCK_PIN 6
5  #define MOSI_PIN 7
6  #define MISO_PIN 8
7  #define CS_PIN 10
8
9  LSM6DSV16XSensor sensor(&SPI, CS_PIN, 10000000);
10
11 // 2-byte sync + 4-byte timestamp + 24-byte data = 30 bytes
12 struct __attribute__((packed)) ImuPacket {
13     uint16_t sync = 0xBBAA;
14     uint32_t timestamp; // Hardware microsecond timestamp
15     int32_t data[6];
16 } packet;
17
18 // --- FILTER STATE VARIABLES ---
19 int32_t filtered_accel[3] = {0, 0, 0};
20 int32_t filtered_gyro[3] = {0, 0, 0};
21
22 // --- FILTER STRENGTH ---
23 const int FILTER_SHIFT = 3;
24
25 // --- DECIMATION COUNTER ---
26 int decimation_counter = 0;
27
28 void setup()
29 {
30     Serial.begin(2000000);
31
32     pinMode(CS_PIN, OUTPUT);
33     digitalWrite(CS_PIN, HIGH);
34     SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, -1);
35
36     sensor.begin();
37     sensor.Enable_X();
```

```

38     sensor.Enable_G();
39
40     sensor.Set_X_ODR(7680.0f);
41     sensor.Set_X_FS(4);
42     sensor.Set_G_ODR(7680.0f);
43     sensor.Set_G_FS(1000);
44 }
45
46 void loop()
47 {
48     uint8_t status = 0;
49     sensor.Get_X_DRDY_Status(&status);
50
51     if (status == 1) {
52         int32_t raw_accel[3];
53         int32_t raw_gyro[3];
54
55         // Read the raw data
56         sensor.Get_X_Axes(raw_accel);
57         sensor.Get_G_Axes(raw_gyro);
58
59         // Apply the high-speed integer EMA filter at the full 7.68 kHz
60         for (int i = 0; i < 3; i++) {
61             filtered_accel[i] += (raw_accel[i] - filtered_accel[i]) >> FILTER_SHIFT;
62             filtered_gyro[i] += (raw_gyro[i] - filtered_gyro[i]) >> FILTER_SHIFT;
63
64             packet.data[i] = filtered_accel[i];
65             packet.data[i+3] = filtered_gyro[i];
66         }
67
68         // Decimate the output to ~1 kHz (send every 7th sample)
69         decimation_counter++;
70         if (decimation_counter >= 7) {
71             packet.timestamp = micros(); // Grab exact hardware time
72             Serial.write((uint8_t*)&packet, sizeof(packet));
73             decimation_counter = 0;
74         }
75     }

```

B Test Setup Hardware and Software

B.1 Hardware Utilized

The following table details the specific hardware components utilized in the planar test setup.

Table 3: Experimental Setup Hardware Components

Component	Specific Part / Model
Tip Tracking Camera	Luxonis Oak 1-Max
Sensor Polling Microcontroller	ESP32-C3-DevKitC-02
Motor Controller	Robotis U2D2 with Power Hub Board
Actuation Motors (x4)	Robotis Dynamixel XL430-W250-T
IMU Sensor IC	Custom carrier board for LSM6DSV16X (STMicro-electronics)
Power Supply	Generic 60W 12V power supply with barrel jack output
Control Computer	ASUS ROG Zephyrus G15 Laptop

B.2 Computer Software

main.py

```

1  #!/usr/bin/env python3
2
3  import numpy as np
4  import multiprocessing
5  import time
6  from multiprocessing import shared_memory
7
8  from tracking_cam import TrackingCam
9  from imu import IMUReader
10 from continuum_robot import CR
11
12 # Constants for creating shared memory of the right size for tip position
13 TIP_POS_SHAPE = (1, 2) # [x, y] array

```

```

14 TIP_POS_DATA_TYPE = np.float64
15 TIP_POS_SIZE = int(np.dtype(TIP_POS_DATA_TYPE).itemsize * np.prod(TIP_POS_SHAPE))
16
17 # Constants for IMU shared memory
18 IMU_SHAPE = (1, 6) # [ax, ay, az, gx, gy, gz]
19 IMU_DATA_TYPE = np.float64
20 IMU_SIZE = int(np.dtype(IMU_DATA_TYPE).itemsize * np.prod(IMU_SHAPE))
21
22 BOT_PORT = '/dev/ttyUSB0'
23 IMU_PORT = '/dev/ttyUSB1'
24
25 def release_shared_mem(name):
26     shm = shared_memory.SharedMemory(name=name)
27     shm.close()
28     shm.unlink()
29
30 def plot_sensor_data():
31     import pandas as pd
32     import matplotlib.pyplot as plt
33
34     colors = ['#0072B2', '#D55E00', '#009E73']
35
36     print("\nGenerating plots...")
37
38     try:
39         df_cam = pd.read_csv('tip_cam.csv')
40         df_imu = pd.read_csv('imu.csv')
41
42         # Sync start time across both sensors using the PC timestamps
43         start_time = min(df_cam['timestamp'].min(), df_imu['pc_timestamp'].min())
44
45         df_cam['time_s'] = df_cam['timestamp'] - start_time
46         df_imu['time_s'] = df_imu['pc_timestamp'] - start_time
47
48         # Create a figure with 3 stacked subplots
49         fig, axes = plt.subplots(3, 1, figsize=(10, 10), sharex=True)
50         fig.suptitle('Continuum Robot Sensor Data', fontsize=16)
51
52         # Plot Tip Position
53         axes[0].plot(df_cam['time_s'], df_cam['x'], label='X Position', color=colors[0])
54         axes[0].axhline(y=0, color='black', linestyle='--', alpha=0.8, label='Joining
↪ Wall')
55         #axes[0].plot(df_cam['time_s'], df_cam['y'], label='Y Position', color='r')
56         axes[0].set_title('Camera Tip Tracking')

```

```

57     axes[0].set_ylabel('mm')
58     axes[0].legend(loc='upper right')
59     axes[0].grid(True, linestyle='--', alpha=0.7)
60
61     # Plot IMU Accelerometer
62     axes[1].plot(df_imu['time_s'], df_imu['accel_x'], label='Accel X',
63     ↪ color=colors[1])
64     #axes[1].plot(df_imu['time_s'], df_imu['accel_y'], label='Accel Y')
65     #axes[1].plot(df_imu['time_s'], df_imu['accel_z'], label='Accel Z')
66     axes[1].set_title('IMU Accelerometer')
67     axes[1].set_ylabel('mgs')
68     axes[1].legend(loc='upper right')
69     axes[1].grid(True, linestyle='--', alpha=0.7)
70
71     # Plot IMU Gyroscope
72     #axes[2].plot(df_imu['time_s'], df_imu['gyro_x'], label='Gyro X')
73     axes[2].plot(df_imu['time_s'], df_imu['gyro_y'], label='Gyro Y', color=colors[2])
74     #axes[2].plot(df_imu['time_s'], df_imu['gyro_z'], label='Gyro Z')
75     axes[2].set_title('IMU Gyroscope')
76     axes[2].set_ylabel('mdps')
77     axes[2].set_xlabel('Time (seconds)')
78     axes[2].legend(loc='upper right')
79     axes[2].grid(True, linestyle='--', alpha=0.7)
80
81     plt.tight_layout()
82     plt.show()
83
84     except Exception as e:
85         print(f"Could not plot data. Did the sensors record correctly? Error: {e}")
86
87 if __name__ == "__main__":
88     multiprocessing.set_start_method("spawn")
89
90     # Creating shared memory to hold data from each process
91     tip_pos_mem = shared_memory.SharedMemory(create=True, size=TIP_POS_SIZE,
92     ↪ name="tip_pos")
93     tip_pos = np.ndarray(shape=TIP_POS_SHAPE, dtype=TIP_POS_DATA_TYPE,
94     ↪ buffer=tip_pos_mem.buf)
95
96     imu_mem = shared_memory.SharedMemory(create=True, size=IMU_SIZE, name="imu_data")
97     imu_data = np.ndarray(shape=IMU_SHAPE, dtype=IMU_DATA_TYPE, buffer=imu_mem.buf)
98
99     stop_event = multiprocessing.Event()

```



```

98     tip_cam = TrackingCam(name="tip_cam", stop_event=stop_event)
99     imu = IMUReader(name="imu", port=IMU_PORT, baudrate=2000000, stop_event=stop_event)
100
101     # Starting processes
102     tip_cam.start()
103     imu.start()
104
105     #
106     ↪ -----
107     motor_ids = np.array([0, 1, 2, 3], dtype=np.uint8)
108     motor_map = np.array([[0, 3], [1, 2]], dtype=np.uint8)
109
110     squiggly = CR(motor_ids, motor_map) # A cute name for a cute boy
111     squiggly.startup(BOT_PORT)
112
113     snap_triggered = False
114
115     join = np.array([120, 0], dtype=np.int32)
116     unjoin = np.array([-170, 0], dtype=np.int32)
117
118     left = np.array([60, 0], dtype=np.int32)
119     right = np.array([-60, 0], dtype=np.int32)
120
121     comp_profile = np.genfromtxt('compensation_profile copy.csv', delimiter=',',
122     ↪ dtype=np.int32)
123     #comp_profile *= -1
124     #print(comp_profile)
125
126     #squiggly.tighten('pos', 25)
127     #squiggly.neutral_pos = squiggly.pos_map
128
129     seg_weights = np.array([0.25, 0.75])
130     prev_accel = 0
131
132     K_a = 0.03
133     K_s = 0.4
134
135     join = np.array([90, 0], dtype=np.int32)
136     unjoin = np.array([-90, 0], dtype=np.int32)
137
138     time.sleep(4)
139     squiggly.actuate_segs('pos', join)
140     time.sleep(2) # Allow for processes to warm up
141     squiggly.actuate_segs('pos', unjoin)

```

```

140     print("Starting Controller")
141     time.sleep(0.05)
142
143     start_time = time.time()
144     while time.time()-start_time < 3:
145         #print(f"Tip Pos: {tip_pos[0]}")
146         #print(f"IMU Data: {imu_data[0]}")
147
148         accel_err = (K_a*imu_data[0][0]*seg_weights).astype(np.int32)
149         seg_err = (K_s * squiggly.get_seg_err()).astype(np.int32)
150
151         u = accel_err + seg_err
152         squiggly.actuate_segs('pos', u)
153         print(accel_err, seg_err)
154         #time.sleep(0.05)
155
156         '''
157         # Compensation Profile Test
158         time.sleep(1)
159         squiggly.actuate_segs('pos', join)
160         time.sleep(1)
161         squiggly.actuate_segs('pos', unjoin)
162         time.sleep(0.1)
163         #squiggly.actuate_profile('pos', comp_profile)
164         squiggly.actuate(neutral)
165         #squiggly.actuate(squiggly.neutral_pos)
166         time.sleep(1)
167         break
168         '''
169
170         '''
171         # IMU Triggering Test
172         if imu_data[0][0] > 1000 and not snap_triggered:
173             snap_triggered = True
174             print("Snap!")
175             squiggly.tighten('pos', 80)
176         '''
177
178     #
179     ↪ -----
180
181     print("Shutting down processes and saving data...")
182     stop_event.set()

```

```

183     tip_cam.join()
184     imu.join()
185     print("Data saved successfully.")
186
187     release_shared_mem("tip_pos")
188     release_shared_mem("imu_data")
189
190     print("Plotting Data")
191     plot_sensor_data()
192

```

imu.py

```

1  #!/usr/bin/env python3
2
3  from multiprocessing import Process, shared_memory
4  import numpy as np
5  import time
6  import serial
7  import struct
8  import csv
9
10 IMU_SHAPE = (1, 6)
11 IMU_DATA_TYPE = np.float64
12
13 class IMUReader(Process):
14     def __init__(self, name, port, baudrate, stop_event):
15         super().__init__(name=name)
16         self.port = port
17         self.baudrate = baudrate
18         self.stop_event = stop_event
19
20     def run(self):
21         imu_mem = shared_memory.SharedMemory(name='imu_data')
22         imu_output = np.ndarray(IMU_SHAPE, dtype=IMU_DATA_TYPE, buffer=imu_mem.buf)
23
24         data_buffer = []
25
26         # Format: '<' (Little Endian), 'H' (Unsigned Short), 'I' (Unsigned Int), '6i' (6
↪ Integers)
27         # Total packet size = 30 bytes
28         packet_format = '<HI6i'
29         packet_size = struct.calcsize(packet_format)

```

```

30
31     byte_buffer = bytearray()
32
33     try:
34         ser = serial.Serial(self.port, self.baudrate, timeout=0)
35     except serial.SerialException as e:
36         print(f"IMUReader Error: Could not connect to {self.port}: {e}")
37         return
38
39     while not self.stop_event.is_set():
40         bytes_waiting = ser.in_waiting
41         if bytes_waiting > 0:
42             byte_buffer.extend(ser.read(bytes_waiting))
43
44             latest_packet_bytes = None
45
46             # Process all complete packets in the buffer
47             while len(byte_buffer) >= packet_size:
48                 # Look for sync word (0xAA 0xBB in little endian)
49                 if byte_buffer[0] == 0xAA and byte_buffer[1] == 0xBB:
50                     # Extract the packet
51                     latest_packet_bytes = byte_buffer[:packet_size]
52                     # Remove it from the buffer
53                     del byte_buffer[:packet_size]
54                 else:
55                     # Realign the stream if sync fails
56                     del byte_buffer[0]
57
58             # If we find a valid packet, push the freshest one
59             if latest_packet_bytes is not None:
60                 unpacked_data = struct.unpack(packet_format, latest_packet_bytes)
61
62                 # unpacked_data[0] is sync, [1] is hw timestamp, [2:8] are the IMU
→ values
63                 hw_micros = unpacked_data[1]
64                 imu_values = [float(val) for val in unpacked_data[2:]]
65
66                 # Update shared memory immediately
67                 imu_output[0, :] = imu_values
68
69                 # Log both PC absolute time and hardware relative time
70                 data_buffer.append([time.time(), hw_micros] + imu_values)
71
72                 # Sleep for 1ms to yield the CPU core.

```

```

73         time.sleep(0.001)
74
75     with open('imu.csv', mode='w', newline='') as csv_file:
76         csv_writer = csv.writer(csv_file)
77         csv_writer.writerow(['pc_timestamp', 'hw_micros', 'accel_x', 'accel_y',
78                               ↪ 'accel_z', 'gyro_x', 'gyro_y', 'gyro_z'])
79         csv_writer.writerows(data_buffer)

```

continuum_robot.py

```

1  #!/usr/bin/env python3
2
3  import numpy as np
4  from dynamixel_sdk import *
5  from keyboard import is_pressed
6  import time
7
8  class CR:
9      def __init__(self, IDs, ID_map, control='pos'):
10         self.IDs = IDs
11         self.ID_map = ID_map
12
13         self.pos_map = np.array([[2048, 2048],[2048, 2048]], dtype=np.int32)
14         self.pwm_map = np.array([[0, 0],[0, 0]], dtype=np.int32)
15         self.neutral_pos = np.zeros((2, 2), dtype=np.int32)
16         self.curvatures = np.array([0.0, 0.0])
17
18         self.control = control
19
20         self.packetHandler = None
21         self.groupSyncWritePos = None
22         self.groupSyncWritePWM = None
23         self.groupSyncRead = None
24         self.portHandler = None
25
26     def startup(self, port):
27         self.portHandler = PortHandler(port)
28         self.packetHandler = PacketHandler(2.0)
29
30         goal_position_address = 116
31         present_position_address = 132
32         goal_pwm_address = 100

```

```

33     data_length_4byte = 4
34     data_length_2byte = 2
35
36     self.groupSyncWritePos = GroupSyncWrite(self.portHandler, self.packetHandler,
37     ↪ goal_position_address, data_length_4byte)
38     self.groupSyncWritePWM = GroupSyncWrite(self.portHandler, self.packetHandler,
39     ↪ goal_pwm_address, data_length_2byte)
40     self.groupSyncRead = GroupSyncRead(self.portHandler, self.packetHandler,
41     ↪ present_position_address, data_length_4byte)
42
43     if self.portHandler.openPort():
44         print("Succeeded to open the port!")
45     else:
46         print("Failed to open the port!")
47         exit()
48
49     if self.portHandler.setBaudRate(3000000):
50         print("Succeeded to change the baudrate!")
51     else:
52         print("Failed to change the baudrate!")
53         exit()
54
55     torque_on_address = 64
56     data = 1
57
58     for id in self.IDs:
59         dxl_comm_result, dxl_error =
60         ↪ self.packetHandler.write1ByteTxRx(self.portHandler, id,
61         ↪ torque_on_address, data)
62         if dxl_comm_result != COMM_SUCCESS:
63             print("%s" % self.packetHandler.getTxRxResult(dxl_comm_result))
64         elif dxl_error != 0:
65             print("%s" % self.packetHandler.getRxPacketError(dxl_error))
66         else:
67             print(f"Dynamixel ID:{id} has been successfully connected")
68
69     for id in self.IDs:
70         dxl_addparam_result = self.groupSyncRead.addParam(id)
71         if dxl_addparam_result != True:
72             print("[ID:%03d] groupSyncRead addparam failed" % id)
73             exit()
74
75     print("MANUAL TENSIONING MODE: Use the keyboard to adjust motors.")
76     print("")

```

```

72     print("0: Q/A")
73     print("1: W/S")
74     print("2: E/D")
75     print("3: R/F")
76     print("")
77     print("Press C to confirm and exit.")
78     print("")
79
80     while True:
81         if is_pressed('q'):
82             self.pos_map[0][0] += 1
83         elif is_pressed('a'):
84             self.pos_map[0][0] -= 1
85         elif is_pressed('w'):
86             self.pos_map[1][0] += 1
87         elif is_pressed('s'):
88             self.pos_map[1][0] -= 1
89         elif is_pressed('e'):
90             self.pos_map[1][1] += 1
91         elif is_pressed('d'):
92             self.pos_map[1][1] -= 1
93         elif is_pressed('r'):
94             self.pos_map[0][1] += 1
95         elif is_pressed('f'):
96             self.pos_map[0][1] -= 1
97         elif is_pressed('c'):
98             break
99         else:
100             pass
101
102     for segment in range(2):
103         for tendon in range(2):
104             target_position = self.pos_map[segment][tendon]
105
106             param_goal_position_pos = [
107                 DXL_LOBYTE(DXL_LOWORD(target_position)),
108                 DXL_HIBYTE(DXL_LOWORD(target_position)),
109                 DXL_LOBYTE(DXL_HIWORD(target_position)),
110                 DXL_HIBYTE(DXL_HIWORD(target_position))
111             ]
112
113             dxl_addparam_result =
114                 ↪ self.groupSyncWritePos.addParam(self.ID_map[segment][tendon],
115                 ↪ param_goal_position_pos)

```

```

114         if not dxl_addparam_result:
115             print("[ID:%03d] groupSyncWritePos addparam failed" %
116                   ↪ self.ID_map[segment][tendon])
117             exit()
118
119         dxl_comm_result = self.groupSyncWritePos.txPacket()
120         if dxl_comm_result != COMM_SUCCESS:
121             print("%s" % self.packetHandler.getTxRxResult(dxl_comm_result))
122
123         self.groupSyncWritePos.clearParam()
124         time.sleep(0.01)
125
126         self.neutral_pos = self.pos_map
127         print("MANUAL TENSIONING COMPLETE")
128         print(f"Neutral Position: {self.neutral_pos}")
129
130     def switch_control(self, new_control):
131         if new_control == self.control:
132             return
133
134         torque_enable_address = 64
135         operating_mode_address = 11
136
137         # Modes: 3 = Position Control, 16 = PWM Control
138         mode_val = 3 if new_control == 'pos' else 16
139
140         for id in self.IDs:
141             dxl_comm_result, dxl_error =
142                 ↪ self.packetHandler.write1ByteTxRx(self.portHandler, id,
143                 ↪ torque_enable_address, 0)
144             if dxl_comm_result != COMM_SUCCESS:
145                 print(f"[ID:{id}] Failed to disable torque: %s" %
146                       ↪ self.packetHandler.getTxRxResult(dxl_comm_result))
147
148         for id in self.IDs:
149             dxl_comm_result, dxl_error =
150                 ↪ self.packetHandler.write1ByteTxRx(self.portHandler, id,
151                 ↪ operating_mode_address, mode_val)
152             if dxl_comm_result != COMM_SUCCESS:
153                 print(f"[ID:{id}] Failed to change mode: %s" %
154                       ↪ self.packetHandler.getTxRxResult(dxl_comm_result))
155
156         for id in self.IDs:

```



```

150         dxl_comm_result, dxl_error =
            ↪ self.packetHandler.write1ByteTxRx(self.portHandler, id,
            ↪ torque_enable_address, 1)
151     if dxl_comm_result != COMM_SUCCESS:
152         print(f"[ID:{id}] Failed to enable torque: %s" %
            ↪ self.packetHandler.getTxRxResult(dxl_comm_result))
153
154     self.control = new_control
155     print(f"Control mode successfully switched to: {self.control}")
156
157     def clip_pwm(self, pwm_array):
158         return np.clip(pwm_array, -885, 885)
159
160     def clip_pos(self, pos_array, min_pos=0, max_pos=4095):
161         return np.clip(pos_array, min_pos, max_pos)
162
163     def actuate(self, map, type='pos'):
164         if type != self.control:
165             self.switch_control(type)
166
167         if type == 'pos':
168             self.pos_map = self.clip_pos(map)
169             for segment in range(2):
170                 for tendon in range(2):
171                     target_position = int(self.pos_map[segment][tendon])
172
173                     param_goal_position_pos = [
174                         DXL_LOBYTE(DXL_LOWORD(target_position)),
175                         DXL_HIBYTE(DXL_LOWORD(target_position)),
176                         DXL_LOBYTE(DXL_HIWORD(target_position)),
177                         DXL_HIBYTE(DXL_HIWORD(target_position))
178                     ]
179
180                     dxl_addparam_result =
                        ↪ self.groupSyncWritePos.addParam(self.ID_map[segment][tendon],
                        ↪ param_goal_position_pos)
181                     if not dxl_addparam_result:
182                         print(f"[ID:%03d] groupSyncWritePos addparam failed" %
                            ↪ self.ID_map[segment][tendon])
183                         exit()
184
185                     dxl_comm_result = self.groupSyncWritePos.txPacket()
186                     if dxl_comm_result != COMM_SUCCESS:
187                         print(f"%s" % self.packetHandler.getTxRxResult(dxl_comm_result))

```

```

188
189         self.groupSyncWritePos.clearParam()
190
191     elif type == 'pwm':
192         self.pwm_map = self.clip_pwm(map)
193
194         for segment in range(2):
195             for tendon in range(2):
196                 target_pwm = int(self.pwm_map[segment][tendon])
197
198                 param_goal_pwm = [
199                     DXL_LOBYTE(DXL_LOWORD(target_pwm)),
200                     DXL_HIBYTE(DXL_LOWORD(target_pwm))
201                 ]
202
203                 dxl_addparam_result =
204                     ↪ self.groupSyncWritePWM.addParam(self.ID_map[segment][tendon],
205                     ↪ param_goal_pwm)
206                 if not dxl_addparam_result:
207                     print("[ID:%03d] groupSyncWritePWM addparam failed" %
208                     ↪ self.ID_map[segment][tendon])
209                     exit()
210
211                 dxl_comm_result = self.groupSyncWritePWM.txPacket()
212                 if dxl_comm_result != COMM_SUCCESS:
213                     print("%s" % self.packetHandler.getTxRxResult(dxl_comm_result))
214
215                 self.groupSyncWritePWM.clearParam()
216
217
218     def read_positions(self):
219         present_position_address = 132
220         data_length_4byte = 4
221
222         # Transmit the SyncRead instruction and receive status packets
223         dxl_comm_result = self.groupSyncRead.txRxPacket()
224         if dxl_comm_result != COMM_SUCCESS:
225             print("%s" % self.packetHandler.getTxRxResult(dxl_comm_result))
226             # Return the last commanded positions as a fallback if communication fails
227             return self.pos_map
228
229         # Using int32 since Dynamixel position registers are 4 bytes
230         actual_positions = np.zeros((2, 2), dtype=np.int32)
231
232         for segment in range(2):

```

```

229         for tendon in range(2):
230             motor_id = self.ID_map[segment][tendon]
231
232             # Verify that the data actually arrived for this specific ID
233             data_available = self.groupSyncRead.isAvailable(motor_id,
234                 ↪ present_position_address, data_length_4byte)
235             if not data_available:
236                 print(f"[ID:{motor_id}] groupSyncRead data not available")
237                 return self.pos_map
238
239             # Extract the 4-byte position data
240             present_pos = self.groupSyncRead.getData(motor_id,
241                 ↪ present_position_address, data_length_4byte)
242             actual_positions[segment][tendon] = present_pos
243
244         return actual_positions
245
246     def actuate_segs(self, type, vals):
247         # +ve val = actuate towards magnets
248         if type == 'pos':
249             self.pos_map = self.read_positions() #idk if this is needed but safe
250             new_map = self.pos_map
251
252             for seg in range(2):
253                 if seg == 0:
254                     new_map[0][0] += vals[seg]
255                     new_map[0][1] -= vals[seg]
256                     new_map[1][0] -= vals[seg]
257                     new_map[1][1] += vals[seg]
258                 else:
259                     new_map[1][0] -= vals[seg]
260                     new_map[1][1] += vals[seg]
261
262             self.actuate(new_map, 'pos')
263
264         elif type == 'pwm':
265             new_map = np.zeros((2, 2), dtype=np.int32)
266
267             for seg in range(2):
268                 if seg == 0:
269                     new_map[0][0] += vals[seg]
270                     new_map[0][1] -= vals[seg]
271                     new_map[1][0] -= vals[seg]
272                     new_map[1][1] += vals[seg]

```

```

271         else:
272             new_map[1][0] -= vals[seg]
273             new_map[1][1] += vals[seg]
274
275         self.actuate(new_map, 'pwm')
276
277     def tighten(self, type, strength):
278         if type == 'pwm':
279             tightening_map = np.zeros((2, 2), dtype=np.int32)
280         elif type == 'pos':
281             self.pos_map = self.read_positions() #idk if this is needed but safe
282             tightening_map = self.pos_map
283
284             tightening_map[0][0] -= strength
285             tightening_map[0][1] -= strength
286             tightening_map[1][0] += strength
287             tightening_map[1][1] += strength
288
289             self.actuate(tightening_map, type)
290
291     def actuate_profile(self, type, profile, spacing_delay=0.05):
292         profile_len = profile.shape[0]
293         for i in range(profile_len):
294             command = profile[i][:]
295             self.actuate_segs(type, command)
296             time.sleep(spacing_delay)
297
298     def get_seg_err(self):
299         error = np.array([0, 0], dtype=np.int32)
300         diff = self.read_positions() - self.neutral_pos
301
302         error[0] = diff[0][1] - diff[0][0]
303         error[1] = diff[1][0] - diff[1][1]
304         error[1] -= error[0]
305
306         return error
307

```

tracking_cam.py

```

1  #!/usr/bin/env python3
2
3  from multiprocessing import Process, shared_memory

```

```

4  import numpy as np
5  import depthai as dai
6  import time
7  import csv
8
9  FRAME_SHAPE = (1,2)
10 FRAME_DATA_TYPE = np.float64
11
12 WALL_PXL = 660
13 IMG_HEIGHT = 400
14 PXL_2_MM = 1/8
15
16 def get_tip_pos(tip_mask):
17     tip_pos = np.zeros((1, 2))
18     y_s, x_s = np.where(tip_mask != 0)
19     if np.size(x_s) != 0:
20         tip_pos[0][0] = (WALL_PXL - np.average(x_s)) * PXL_2_MM
21         tip_pos[0][1] = (IMG_HEIGHT - np.average(y_s)) * PXL_2_MM
22     return tip_pos
23 else:
24     return None
25
26
27 class TrackingCam(Process):
28     def __init__(self, name, stop_event):
29         super().__init__(name=name)
30         self.stop_event = stop_event
31
32     def run(self):
33         import cv2
34         import depthai as dai
35         import csv
36         import time
37
38         tip_pos_mem = shared_memory.SharedMemory(name='tip_pos')
39         tip_pos_output = np.ndarray(FRAME_SHAPE, dtype=FRAME_DATA_TYPE,
40                                     ↪ buffer=tip_pos_mem.buf)
41
42         data_buffer = []
43
44         with dai.Pipeline() as pipeline:
45             cam = pipeline.create(dai.node.Camera)
46             cam.build(dai.CameraBoardSocket.CAM_A, (3840, 2160), 42)

```

```

47     # Set the manual focus (0-255)
48     cam.initialControl.setManualFocus(180)
49
50     # Request the output, scaled down by the ISP to 720p at 42 FPS
51     videoQueue = cam.requestOutput((1280, 720), fps=42).createOutputQueue()
52     pipeline.start()
53
54     while pipeline.isRunning() and not self.stop_event.is_set():
55         videoIn = videoQueue.tryGet()
56         if videoIn is not None:
57             frame = videoIn.getCvFrame()
58             frame = frame[0:400, 300:1000]
59
60             hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
61             lower_hsv = np.array([10, 110, 100])
62             upper_hsv = np.array([20, 255, 255])
63             mask = cv2.inRange(hsv, lower_hsv, upper_hsv)
64
65             tip_pos = get_tip_pos(mask)
66
67             if tip_pos is not None:
68                 tip_pos_output[:] = tip_pos
69                 data_buffer.append([time.time(), tip_pos[0][0], tip_pos[0][1]])
70
71     # Write data from buffer to CSV file
72     with open('tip_cam.csv', mode='w', newline='') as csv_file:
73         csv_writer = csv.writer(csv_file)
74         csv_writer.writerow(['timestamp', 'x', 'y'])
75         csv_writer.writerows(data_buffer)
76

```
